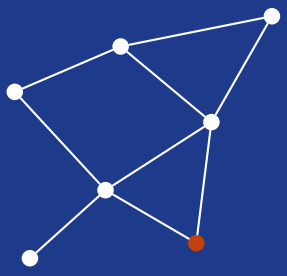
Competitive Programming Setup



User Guide · VS Code + CPH

Everything installed in `Desktop/competitive`: the template, the 175-template library and its keyboard shortcuts, the CPH integration, the test runner, the stress tester and the tools behind them, and how to use every piece of it.

Kshitish Madbhavi

Companion volume: *The Competitive Programming Handbook* (ALGORITHMS.pdf)

Contents

1 Overview **1**

1.1 The big picture . . . 1
1.2 Quick start in 60 seconds . . . 1
1.3 What is where . . . 2
1.4 What was changed outside the folder . . . 2
1.5 The four shortcuts that were broken before . . . 3
1.6 How this setup is tied to CPH . . . 3

2 The Daily Workflow **4**

2.1 Solving a problem, step by step . . . 4
2.2 Keyboard shortcuts . . . 6
2.3 Multi-test problems . . . 6
2.4 USACO problems . . . 6
2.5 Quickstart and archiving . . . 7

3 The Template, Line by Line **8**

3.1 Header . . . 8
3.2 Includes and the debug hook . . . 8
3.3 Types . . . 9
3.4 Macros . . . 9
3.5 Constants . . . 10
3.6 Helpers . . . 10
3.7 Automatic USACO file I/O . . . 11
3.8 solve() and main() . . . 12

4 Testing and Debugging **13**

4.1 Debug printing with dbg() . . . 13
4.2 F5: CPH's tests with diagnostics . . . 14
4.3 Ctrl+F5: typing input yourself . . . 15
4.4 Stress testing . . . 15

5 Compiler Flags and Speed **17**

5.1 What each flag does . . . 17
5.2 The precompiled header . . . 17
5.3 The optional UB trap . . . 18
5.4 Time limits and the checked STL . . . 18

6 The Template Library **19**
6.1 Three ways to insert a template . . . 19
6.2 Where to paste . . . 19
6.3 Conventions used everywhere . . . 20
6.4 Keyboard maps . . . 20
6.5 Categories . . . 22

7 Maintenance and Troubleshooting **23**
7.1 VS Code tasks . . . 23
7.2 CP: Setup . . . 23
7.3 Adding your own template . . . 23
7.4 Rebuilding these PDFs . . . 24
7.5 Troubleshooting . . . 24
7.6 Undoing everything . . . 25

8 Library Reference **26**
8.1 Template . . . 26
8.2 Basics: input, search, sorting, prefix sums, two pointers . . . 26
8.3 Graphs . . . 30
8.4 Trees . . . 34
8.5 Data structures & range queries . . . 37
8.6 Dynamic programming . . . 41
8.7 Math . . . 44
8.8 Strings . . . 50
8.9 Geometry . . . 52
8.10 Flows & matching . . . 53
8.11 Misc . . . 55

CHAPTER 1

Overview

This guide documents the competitive programming environment set up in the `Desktop/competitive` folder. It is built around one idea: **the CPH extension stays in charge of test cases**. You fetch a problem, CPH creates the file from your template and stores the samples, you write code, and you press CPH's *Run All* button. Everything else in this setup either makes that loop faster or explains what CPH cannot.

1.1 The big picture

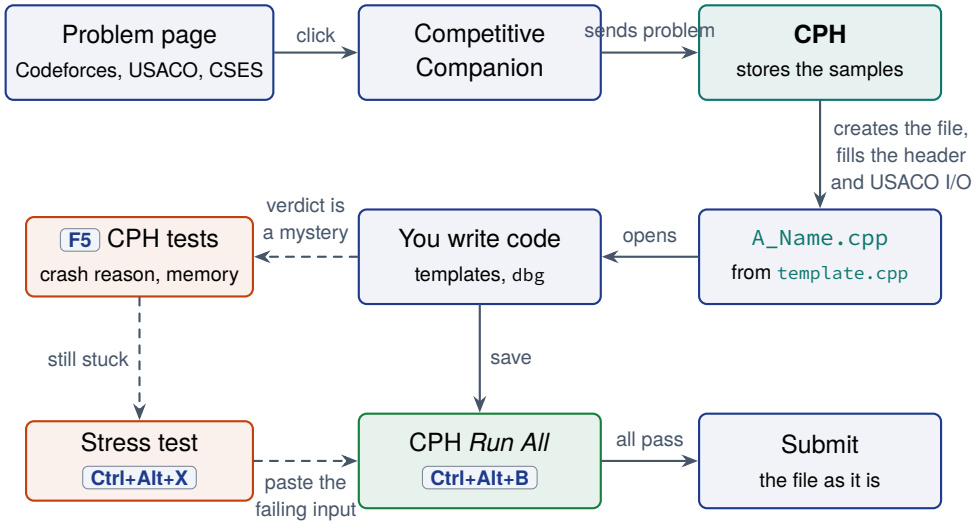


Figure 1.1: The daily loop. Solid arrows are the normal path; dashed arrows are what you do when a verdict surprises you.

1.2 Quick start in 60 seconds

1. Open the `competitive` folder in VS Code. It uses the *competitive* profile automatically.
2. On a problem page, click the green Competitive Companion button. CPH creates `A_Problem_Name.cpp` from the template, fills in the problem name, link and time, and (for USACO) switches on file I/O.
3. Write your solution in `solve()`. Type `cp` to see every library snippet, or press Ctrl+Alt+T followed by a key (for example U for a DSU) to paste one at the cursor.
4. Use `dbg(x, v)` freely: it prints to CPH's *Standard Error* box and vanishes on the judge.
5. Press Ctrl+Alt+B (or CPH's *Run All* button). If a verdict is confusing, press F5: the same tests run in the terminal with the crash reason, time and peak memory of each one.
6. When all tests pass, submit the file exactly as it is. Nothing needs to be deleted or uncommented.

> **Tip** The one-page summary of every shortcut is `CHEATSHEET.md` in the folder root. This guide is the long version; the algorithms behind the templates are taught in the companion handbook, *ALGORITHMS.pdf*.

1.3 What is where



Figure 1.2: The folder layout. Build output (`.build/`, `*.exe`, `*.bin`) and `.cph/` are hidden from VS Code's explorer.

1.4 What was changed outside the folder

Keyboard shortcuts and snippets can only live in VS Code's user settings, so a few files outside the folder were changed. All of them belong to the *competitive* VS Code profile only; your default profile is untouched.

File (in the profile folder)	What changed
`keybindings.json`	One generated block, between »» `cp-library shortcuts` and «« markers, with the Ctrl+Alt+T / Ctrl+Alt+Y / Ctrl+Alt+U shortcuts, Ctrl+Alt+X and Ctrl+F5. Your own bindings above it are untouched. Backup: `keybindings.json.bak-cp-setup`.
`snippets/cp-library.code-snippets`	New file: the 175 snippets.
`settings.json`	Four CPH settings: compiler flags, template path, template variables on, stderr ignored for verdicts. Backup: `settings.json.bak-cp-setup`.
TinyTeX	LaTeX packages were added (fonts, listings, forest, . . .) to compile these PDFs. Additive only.

The profile folder is `%APPDATA%/Code/User/profiles/5d3986b6`. Re-running *CP: Setup* rewrites only the generated block and the four CPH settings, so it is always safe to run again.

1.5 The four shortcuts that were broken before

Your profile already had F5, Ctrl+Shift+N, Ctrl+Shift+U and Ctrl+Shift+Alt+C bound to tasks named `cf-run`, `CP: Quickstart`, `CP: Quickstart USACO` and `CP: Archive Old Files`, but no such tasks existed anywhere, so the keys did nothing. They are now defined in `.vscode/tasks.json` and work as described in Chapter 4 and Section 2.5.

1.6 How this setup is tied to CPH

CPH is the centre of the workflow, so every piece was checked against CPH's own source code:

- **CPH compiles with your flags.** CPH runs `g++ file.cpp -o file.bin` *Args* `-D DEBUG -D CPH`. The *Args* setting is written by *CP: Setup* from one list in `tools/cp.py`, so the precompiled header, the 256 MB stack, static linking and `-DLOCAL` all apply to CPH runs. (The extra `-D DEBUG -D CPH` were verified not to disturb the precompiled header.)
- **CPH fills the template.** With template variables enabled, CPH replaces `$name$`, `$group$`, `$url$`, `$date$`, `$time$`, `$input$` and `$output$` when it creates a file. That is how the header and the USACO file I/O are filled in.
- **CPH shows `dbg` output.** CPH treats any stderr output as a failed test unless `ignoreSTDERROR` is on. It is now on, so `dbg` output appears in the *Standard Error* box while verdicts depend only on stdout and the exit code.
- **F5 reads CPH's tests.** The terminal runner reads the same `.cph/*.prob` file CPH uses.
- **The stress tester checks your brute force against CPH's samples** before it trusts it, and puts a failing input on the clipboard ready to paste into CPH as a new test case.

Honest limits Nothing is lost when you use only the CPH button. What CPH cannot do by design: tell you *why* a program crashed (it just fails the test), show memory use, or accept test cases from outside tools while its panel is open. F5 covers the first two; the clipboard hand-off covers the third. The checked STL (`-D_GLIBCXX_DEBUG`) makes local running times slower than the judge's, so treat CPH's times as upper bounds.

CHAPTER 2

The Daily Workflow

2.1 Solving a problem, step by step

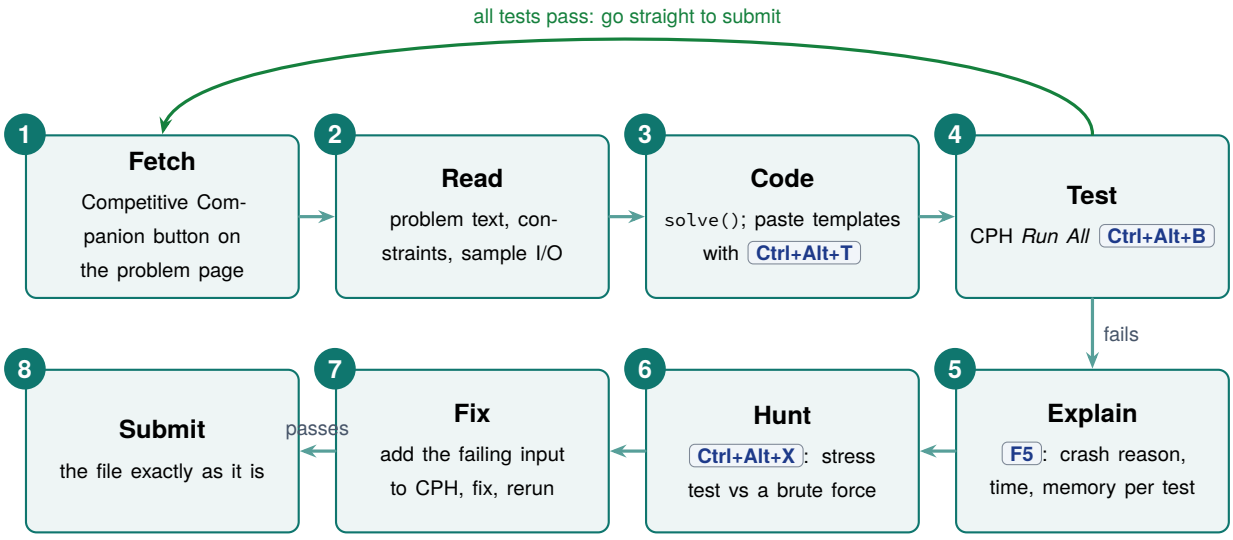


Figure 2.1: Steps 5–7 are only needed when a test fails and the reason is not obvious.

2.1.1 1. Fetch

Click the Competitive Companion button in the browser. CPH receives the problem (name, contest, link, samples, limits and whether it reads from a file) and creates `A_Problem_Name.cpp` from `template/template.cpp`. The file opens with its header already filled in:

```
/**
 *    author:  Kshitish Madbhavi
 *    problem: A. Graph
 *    source:  Codeforces - Baltic Olympiad in Informatics 2020, Day 2 (IOI, Unofficial
    Mirror Contest, Unrated)
 *    url:     https://codeforces.com/contest/1387/problem/A
 *    created: 2026-09-29 20:31:55
 */
```

For a USACO problem the same step also switches on file input and output; you do not touch anything (Section 2.4).

2.1.2 2. Read

Look at the constraints before thinking about an algorithm: they tell you the complexity you can afford (ALGORITHMS.pdf, Chapter 2 has the full table). Decide whether the input has several test

cases. If it starts with "the first line contains t", uncomment `cin >> t;` in `main` (put the cursor on that line and press Ctrl+/).

2.1.3 3. Code

Write the solution in `solve()`. Three ways to pull in library code, all of which paste at the cursor:

Method	Example
Chord shortcut	Ctrl+Alt+T then J pastes Dijkstra; Ctrl+Alt+Y then H pastes HLD.
Prefix + Tab	type `cpdsu` and press Tab. Every prefix starts with `cp`, so typing `cp` lists all of them.
Picker	Ctrl+Alt+T then Space: a searchable list of every template.

Chapter 6 explains the library and has keyboard maps of every chord.

2.1.4 4. Test

Press Ctrl+Alt+B or CPH's *Run All* button. CPH compiles with the flags from Chapter 5 (about 0.4–0.9 s thanks to the precompiled header) and runs every sample. `dbg` output appears in each test's *Standard Error* box without affecting the verdict. Add your own tests with CPH's *New Testcase* button; edge cases (smallest n, largest n, all equal values, negative numbers) catch most wrong answers.

2.1.5 5–7. When a test fails

- **Wrong answer and you can see why**: fix it and rerun.
- **Runtime error, or a time you do not understand**: press F5. The same CPH tests run in the terminal and each failure shows the input, expected output, your output, your stderr and a plain-language crash reason such as "stack overflow" (Section 4.2).
- **The samples pass but the judge says wrong answer**: write a slow brute force in `stress/brute.cpp` and a random input generator in `stress/gen.cpp`, then press Ctrl+Alt+X. The stress tester finds a small input where your answer differs and copies it to the clipboard; paste it into CPH as a new test (Section 4.4).

2.1.6 8. Submit

Submit the file unchanged: `dbg` lines compile to nothing on the judge, and USACO file I/O is already on. CPH's *Submit* (Ctrl+Alt+S) sends Codeforces submissions as *GNU G++23 14.2 (64 bit)*. Your code is still strictly C++17: that is what CPH and IntelliSense compile it as, and every library template is checked with `-std=c++17 -Wpedantic`. G++23 was kept because it is the newest 64-bit compiler Codeforces offers and accepts C++17 code unchanged.

2.2 Keyboard shortcuts

Keys	Action
Ctrl+Alt+B	CPH: compile and run all test cases (CPH's own shortcut).
Ctrl+Alt+S	CPH: submit to Codeforces.
Ctrl+Alt+T, *key*	paste a core template (Bronze to Gold level).
Ctrl+Alt+Y, *key*	paste an advanced template (Platinum and beyond).
Ctrl+Alt+U, *key*	paste a specialist template (polynomials, matchings, cuts, exotic structures).
Ctrl+Alt+T, Space	searchable template picker.
F5	run the CPH test cases in the terminal with diagnostics.
Ctrl+F5	run with input you type in the terminal.
Ctrl+Alt+X	stress test the current file.
Ctrl+Shift+N	new problem file from the template (asks for a name).
Ctrl+Shift+U	new USACO problem (asks for the file name, e.g. `moocast`).
Ctrl+Shift+Alt+C	archive solutions untouched for a day into `archive/YYYY-MM/`.
Ctrl+Alt+N	Code Runner: same as Ctrl+F5.

2.3 Multi-test problems

Competitive Companion does not tell CPH whether a problem has several test cases (it always reports "single"), so this cannot be detected automatically. The template therefore reads one test by default:

```
int t = 1;
// cin >> t;
while (t--) solve();
```

For a multi-test problem, uncomment the middle line. Remember that everything `solve()` uses must be reset between test cases: declare arrays inside `solve()` (sized by that test's n) rather than globally.

> **⚠ Resetting global state** The most common multi-test bug is a global array or counter that keeps values from the previous test. Either declare it inside `solve()` or clear exactly the part you used; never `memset` a 10^6-element array 10^5 times, which is 10^{11} operations.

2.4 USACO problems

USACO reads from `name.in` and writes `name.out`. When Competitive Companion sends a USACO problem it tells CPH the file names, and CPH writes them into the template (Section 3.7 shows exactly how). On the USACO judge your program opens the files; locally, CPH and F5 keep feeding it standard input, so the samples still run in CPH. There is nothing to uncomment.

For a USACO problem typed in by hand, press Ctrl+Shift+U and enter the file name (`moocast`): the new file behaves exactly like one created by CPH.

2.5 Quickstart and archiving

- Ctrl+Shift+N asks for a name, turns it into a safe file name (`B. Two Sum` becomes `B_Two_Sum.cpp`), fills the template exactly as CPH would and opens the file. Use it for problems without Competitive Companion support. If the file exists it is simply opened.
- Ctrl+Shift+U does the same for USACO and fills in `name.in` / `name.out`.
- Ctrl+Shift+Alt+C moves every solution in the folder root that has not been modified for a day into `archive/YYYY-MM/` (by the month it was last edited), deletes stray `.exe` / `.bin` files, and moves each problem's CPH test cases along with it. CPH names a problem's test file after the MD5 hash of the source path, so the test file is renamed for the new path and CPH still finds your tests if you open the archived file. Nothing is ever deleted except build output.

CHAPTER 3

The Template, Line by Line

`template/template.cpp` is the file CPH copies for every new problem. It is strictly C++17, compiles without a single warning under `-Wall -Wextra -Wshadow -Wpedantic`, and was tested three ways: as CPH builds it locally, as a C++17 judge builds it, and as a C++20 judge builds it. This chapter walks through it top to bottom. Your original template is kept as `template/template.cpp.bak`.

3.1 Header

```
/**
 *    author:  Kshitish Madbhavi
 *    problem: $name$
 *    source:  $group$
 *    url:     $url$
 *    created: $date$ $time$
 */
```

The `$...$` words are CPH template variables. When CPH creates a file it replaces each one with the corresponding field of the problem it received, so every solution records where it came from. Files made with Ctrl+Shift+N are filled the same way by `tools/cp.py`; the `cptemplate` snippet fills them with VS Code's date and file-name variables.

3.2 Includes and the debug hook

```
#include <bits/stdc++.h>
using namespace std;

#ifdef LOCAL
#include "debug.h"
#else
#define dbg(...)
#endif
```

`LOCAL` is defined by every local build (CPH, F5, the stress tester) and never by a judge. Locally, `include/debug.h` provides `dbg(...)`; on the judge `dbg(...)` is defined as nothing, so `dbg(a, b);` becomes an empty statement. Chapter 4 shows what `dbg` prints.

3.3 Types

```
#define int long long

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vvi = vector<vi>;
template <class T> using minpq = priority_queue<T, vector<T>, greater<T>>;
```

`#define int long long` turns every `int` in your code into a 64-bit integer, which removes the most common source of wrong answers: overflow of intermediate values such as `a * b` when both are up to 10^9.

> **Fixed from the old template** In the old template `#define int long long` came *after* the aliases, so `pii` was still `pair<int,int>` with 32-bit ints. Pushing a `long long` distance into a `minpqpii` silently truncated it, a classic source of mysterious Dijkstra failures. The define now comes first, so every alias is 64-bit.

Consequences of `#define int long long` to remember:

- `main` must be declared as `signed main()` (it is), because `long long main()` is illegal.
- `printf("%d", x)` is wrong for an `int`; use `cout` or `%lld`.
- `max(x, 0)` fails to compile when `x` is `int`, because `0` is a real 32-bit `int` literal. Write `max<int>(x, 0)`, `max(x, 0LL)` or use `chmax` below (it accepts mixed types).
- `__builtin_popcount(x)` truncates to 32 bits; use `__builtin_popcountll`.
- Memory doubles for `vector<int>`. 10^7 `int`s are now 80 MB instead of 40 MB.

`minpq<T>` is a min-heap: `minpq<pii> pq; pq.push({dist, v});` pops the smallest pair first. It replaces the four separate `minpqi`/`minpql`/`minpqpii`/`minpqpll` aliases of the old template.

3.4 Macros

```
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define rep(...)   // rep(i, n): 0..n-1   rep(i, a, b): a..b-1
#define per(...)   // same ranges, backwards
```

You write	It means	Typical use
`sort(all(v))`	`sort(v.begin(), v.end())`	any STL algorithm on a whole container
`sort(rall(v))`	`sort(v.rbegin(), v.rend())`	sort in decreasing order
`sz(v)`	`(int)v.size()`	avoids signed/unsigned warnings and `v.size() - 1` underflow
`rep(i, n)`	`for (int i = 0; i < n; i++)`	the everyday loop
`rep(i, 1, n + 1)`	`for (int i = 1; i < n + 1; i++)`	1-indexed loops
`per(i, n)`	`for (int i = n - 1; i >= 0; i--)`	reverse loops
`per(i, a, b)`	`for (int i = b - 1; i >= a; i--)`	reverse over $[a, b)$

`rep` picks its two- or three-argument form by counting arguments (the `PICK3` helper macro), a standard C++17-compatible preprocessor trick.

> **Why `sz` matters** `v.size()` is unsigned. If `v` is empty, `v.size() - 1` is $2^{64} - 1$, so `for (int i = 0; i < v.size() - 1; i++)` runs forever. `sz(v) - 1` is -1 and the loop is skipped. Library code avoids the `sz` name for anything else, because the macro would swallow it.

3.5 Constants

```
constexpr ll INF = 4e18;
constexpr int MOD = 1'000'000'007;  // 998'244'353
```

`INF` is large enough to mean "unreachable" yet `INF + INF` still fits in a `long long` ($8\times10^{18} < 9.22\times10^{18}$). `MOD` is used by every modular template in the library (`cpmint`, `cpncr`, ...); switch the comment when a problem wants 998 244 353.

3.6 Helpers

```
chmin(a, b);  chmax(a, b);   // a = min(a, b) / max(a, b); true if a changed; mixed
    types OK
cdiv(a, b);   fdiv(a, b);    // ceil(a / b) and floor(a / b), correct for negative
    numbers too
cin >> v;                    // reads a whole vector (sized beforehand) or a pair
cout << v;                   // prints a vector separated by spaces
print(a, b, c);              // prints "a b c" and a newline
yn(ok);                      // prints YES or NO
```

expression	C++ /	cdiv	fdiv
7 / 2	3	4	3
-7 / 2	-3	-3	-4
7 / -2	-3	-3	-4
-8 / -3	2	3	2

C++ division truncates toward zero, which is neither floor nor ceiling for negative results; `cdiv` and `fdiv` were checked on all sign combinations. `chmin`/`chmax` take two type parameters so `chmax(ans, 0)` compiles even though `0` is a plain `int`.

3.7 Automatic USACO file I/O

```
void file_io() {
#ifndef LOCAL
    auto file = [](const string& spec) { /* extract "fileName":"..." */ };
    string in = file(R"($input$)"), out = file(R"($output$)");
    if (!in.empty()) freopen(in.c_str(), "r", stdin);
    if (!out.empty()) freopen(out.c_str(), "w", stdout);
#endif
}
```

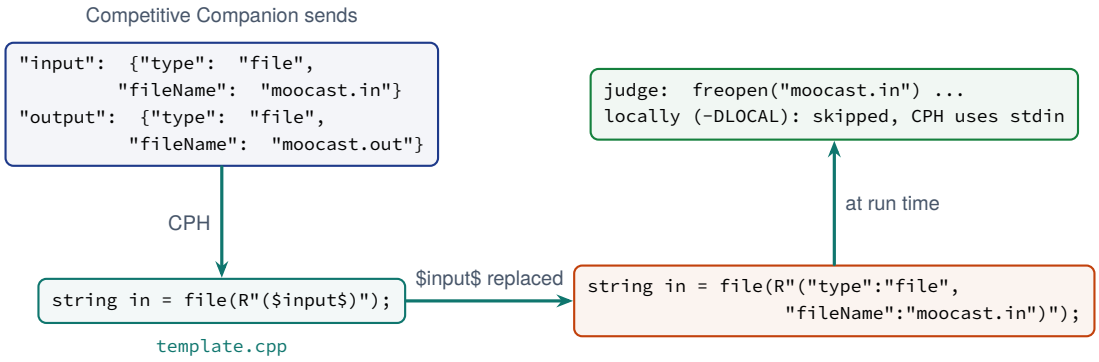


Figure 3.1: How a USACO problem switches on file I/O without any manual step.

CPH replaces `$input$` with the problem's input description, minus its outer braces. For USACO that is `"type":"file","fileName":"moocast.in"`, so `file()` finds a file name and `freopen` runs; for every other judge it is `"type":"stdin"`, no file name is found and nothing happens. The raw string literal `R"(...)"` means the quotes need no escaping. The whole function is compiled only when `LOCAL` is *not* defined, so CPH keeps running your samples through standard input.

> **Tip** If a file was created some other way and you need file I/O, change the placeholder by hand to `R"("fileName":"name.in")"` (and the output one to `name.out`), or recreate the file with Ctrl+Shift+U.

3.8 solve() and main()

```
void solve() {

}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout << fixed << setprecision(10);
    file_io();

    int t = 1;
    // cin >> t;
    while (t--) solve();
}
```

- `sync_with_stdio(false)` and `cin.tie(nullptr)` make `cin`/`cout` as fast as `scanf`. After them, do not mix `cin` with `scanf` or `cout` with `printf`.
- `fixed << setprecision(10)` prints floating-point answers with ten decimals, which satisfies every "absolute or relative error 10^{-6}" checker. Integers are unaffected.
- Print `'\n'`, not `endl`: `endl` flushes the output every time and can turn a fast program into a TLE. The one exception is interactive problems, which need the flush (`cpinteractive`).

CHAPTER 4

Testing and Debugging

4.1 Debug printing with dbg()

`dbg(...)` prints any number of expressions with their names and the line number, to standard error:

```
int x = 3; vector<int> v{1, 2};
map<string, pair<int, char>> mp{{"k", {7, 'z'}}};
dbg(x, v, mp);
dbg(v[0] + v[1]);
dbg();                                  // just "reached": is this line executed?
```

```
[L42] x = 3 | v = {1, 2} | mp = {("k", (7, 'z'))}
[L43] v[0] + v[1] = 3
[L44] reached
```

Type	Printed as
numbers, `__int128`	`42`, `-126765060022822940149670320537 6`
`bool`, `char`, strings	`true`, `'z'`, `"text"`
pairs and tuples	`(1, "s", 2.5)`
any container, nested	`{{1, 2}, {3}}`; maps as `{(key, value), ...}`
`vector<bool>`	`{true, false}`
`stack`, `queue`, `priority_queue`	contents from the top/front, the original is untouched
`bitset`	`0101`

Where the output appears: in CPH it is the *Standard Error* box under each test; with F5 it is printed after the test's output when that test fails; with Ctrl+F5 it goes straight to the terminal. The line is flushed immediately, so the last `dbg` before a crash is always visible, which makes `dbg()` a quick way to find the line that crashes.

> **Why dbg never fails a test in CPH** CPH marks a test as failed whenever the program writes anything to standard error, unless the `cph.general.ignoreSTDERROR` setting is on. *CP: Setup* turns it on. Verdicts then depend on the output and the exit code only, and crashes are still caught through the exit code.

> **Tip** `dbg` is compiled out on the judge, so you never need to delete debug lines before submitting. Heavy `dbg` calls inside a hot loop still slow down local runs, so comment those out when timing.

4.2 F5: CPH's tests with diagnostics

F5 (task `cf-run`) runs `tools/cp.py test` on the current file. It reads the same test cases CPH shows, from `.cph/.<file>_<hash>.prob`, compiles with CPH's flags and runs every test with a timeout:

```
[cp] compiled tricky.cpp in 0.4 s
[cp] tricky.cpp: 3 CPH test case(s), time limit 1000 ms
[cp] test 1: PASS  153 ms, 0.9 MB
[cp] test 2: WRONG ANSWER  7 ms, 0.9 MB
---- input ----
2
5 5
---- expected ----
10
---- got ----
11
---- stderr ----
debug: n=2
[cp] test 3: RUNTIME ERROR, exit 0xC00000FD: stack overflow (recursion too deep or a
    huge local array)  129 ms, 257.3 MB
[cp] 1/3 passed
```

Compared with CPH's panel you get the reason for a crash, the peak memory of every test, and a warning when a test takes longer than the problem's time limit. If the file has no CPH tests yet, F5 falls back to Ctrl+F5 behaviour. Output is compared token by token, so extra spaces or a missing final newline never count as wrong.

Exit code	Meaning and usual cause
`0xC00000FD`	stack overflow: recursion deeper than the 256 MB stack allows, or a huge local array (move it to a global or a vector)
`0xC0000005`	access violation: out-of-bounds pointer or C-array access, dereferencing `end()`
`0xC000001D`	undefined behaviour trapped (only with `UB_TRAP = True`, Section 5.3): signed overflow, bad index, bad shift
`0xC0000094`	integer division or modulo by zero
`0xC0000374`	heap corruption: writing past the end of an array
`3`	`abort()`: a failed `assert`, or the checked STL caught a bug; its message is in stderr

The checked STL Every local build defines `_GLIBCXX_DEBUG`, so `v[i]` with `i` out of range, an invalid comparator in `sort`, or using an invalidated iterator stops the program with a message naming the problem instead of silently producing garbage. That message appears in stderr, i.e. in CPH's Standard Error box or F5's output.

4.3 Ctrl+F5: typing input yourself

Ctrl+F5 compiles the file and runs it in a terminal where you type the input. Finish with Ctrl+Z then Enter (end of file). When the program exits you get its exit code (decoded), CPU time and peak memory:

```
[cp] running sol.cpp  (type the input; Ctrl+Z then Enter sends EOF)
3
1 -2 5
4
[cp] exit 0 | cpu 0 ms | peak memory 0.9 MB | wall 4.21 s
```

Binaries are cached in `.build/`: running again without editing the file skips compilation.

4.4 Stress testing

When the samples pass but the judge disagrees, the bug hides in a case you have not thought of. A stress test finds it automatically by comparing your solution with a slow but obviously correct brute force on thousands of small random inputs.

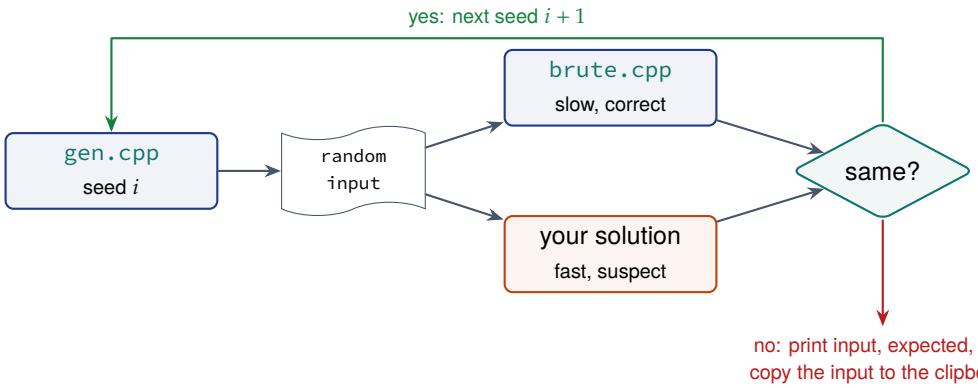


Figure 4.1: The stress loop run by Ctrl+Alt+X.

4.4.1 How to use it

1. Edit `stress/gen.cpp` to print one random test in the problem's input format. The test number arrives as `argv[1]` and seeds the generator, so a failure can be reproduced. Keep inputs *tiny* ($n \leq 8$): small failing cases are easy to understand.
2. Edit `stress/brute.cpp`: the simplest correct solution you can write, however slow (try everything).
3. Open your solution file and press Ctrl+Alt+X.

The tester first runs `brute.cpp` on the problem's CPH samples; if the brute force fails a sample, it says so and stops, because every later verdict would be meaningless. Then it runs 300 random tests (`-n` changes the number when run by hand). On the first difference it prints the input, the brute force's output and yours, saves the input to `stress/fail.in` and copies it to the clipboard. In CPH press *New Testcase*, paste, type the expected output, fix the bug and rerun.

```
[cp] brute.cpp passes all 1 CPH sample(s)
[cp] stress testing positives_only.cpp against brute.cpp (100 tests)
[cp] FAILED on test 1: wrong answer
---- input ----
2
-8 -1
---- expected (brute) ----
-9
---- got (yours) ----
0
[cp] input copied to the clipboard: in CPH press '+ New Testcase', paste it, and use
    the expected output above
```

4.4.2 Problems with several correct answers

If any valid answer is accepted, comparing outputs is wrong. Add `stress/checker.cpp`: it receives the paths of the input, the brute force's output and your output as `argv[1..3]` and must exit with code 0 when your output is valid. When the file exists, the tester uses it instead of comparing text.

> **Tip** Write the brute force and the generator *before* you are desperate. Ten minutes spent on them usually beats an hour of staring at code, and the same pair can be reused for the next attempt.

CHAPTER 5

Compiler Flags and Speed

Every local build (CPH, F5, Ctrl+F5, the stress tester, Code Runner) uses the same flags, defined once in `tools/cp.py` and copied into CPH's settings by *CP: Setup*:

```
-std=c++17 -O2 -DLOCAL -Wall -Wextra -Wshadow -D_GLIBCXX_DEBUG
-IC:/Users/kshit/Desktop/competitive/include -static -Wl,--stack,268435456
```

5.1 What each flag does

Flag	Why it is there
`-std=c++17`	The language standard you asked for. IntelliSense uses it too, so C++20 features are flagged while you type.
`-O2`	The optimisation level judges use; timings are meaningful.
`-DLOCAL`	Defines `LOCAL`: turns on `dbg` and turns off USACO file I/O locally.
`-Wall -Wextra -Wshadow`	Warnings for common bugs: unused variables, sign comparisons, a local that hides another.
`-D_GLIBCXX_DEBUG`	The checked STL: out-of-range indexing, invalid comparators and bad iterators abort with a message.
`-I.../include`	Where `debug.h` and the precompiled `bits/stdc++.h` live.
`-static`	The program carries its own C++ runtime. Other MinGW installs on your PATH (Git's, cpeditor's) can no longer make it load the wrong DLL and die silently.
`-Wl,--stack,268435456`	A 256 MB stack, like Codeforces. Windows' default 2 MB stack crashes a recursive DFS on a 200 000-vertex path; this was measured before and after.

CPH additionally passes `-D DEBUG -D CPH`, so `#ifdef CPH` works if you ever want CPH-only code.

5.2 The precompiled header

Every compile used to spend most of its time parsing `bits/stdc++.h`, which pulls in the entire standard library. `include/bits/` now holds a copy of that header plus `stdc++.h.gch`, the same header pre-parsed with exactly the flags above. Because `-I` puts `include/` first in the search path, GCC finds the `.gch` and loads it in a fraction of the time.

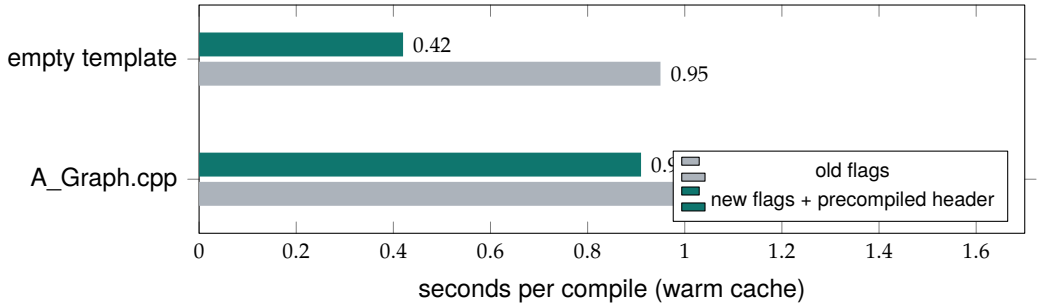


Figure 5.1: Measured CPH compile times. A cold cache (first compile after a reboot) went from about 3.5 s to under 1 s.

> **⚠ Keep CPH's flags in sync** A precompiled header is only valid for the flags it was built with, and GCC does not always reject a mismatched one: during setup a mismatch made GCC crash with an internal error instead of falling back. So never edit CPH's *Args* by hand. Change `COMPILE_FLAGS` in `tools/cp.py` and run *CP: Setup*; it rebuilds the header and rewrites CPH's setting together. F5 and the stress tester also rebuild the header automatically whenever the compiler or the flags change.

5.3 The optional UB trap

`tools/cp.py` has a switch:

```
UB_TRAP = False
```

Set to `True`, it adds `-fsanitize=undefined -fsanitize-undefined-trap-on-error`: signed overflow, out-of-bounds C-array indexing and invalid shifts crash with exit code `0xC000001D` instead of silently producing a wrong answer. It was tested and works with static linking, but costs about 0.35 s per compile, which cancels most of the precompiled header's gain, so it is off by default. With `#define int long long` and the checked STL most of what it catches is already covered. Turn it on (then run *CP: Setup*) when you suspect overflow.

5.4 Time limits and the checked STL

The checked STL verifies every vector access, which can make STL-heavy code several times slower locally than on the judge. If CPH or F5 reports a time close to the limit, that is usually not a real TLE. To measure true speed, temporarily remove `-D_GLIBCXX_DEBUG` from `COMPILE_FLAGS` and run *CP: Setup*; put it back afterwards.

CHAPTER 6

The Template Library

`library/` holds 174 templates plus the full file template, 175 snippets in all, covering the USACO Bronze to Platinum syllabus and the advanced topics beyond it. The second edition of the handbook added 45 of them (discrete logarithms, polynomial tools, FWHT, Du's sieve and prime counting, Hungarian and blossom matching, Gomory–Hu and Stoer–Wagner cuts, the eertree, Lyndon factorization, CDQ, Mo on trees and with updates, persistent treaps, Euler tour trees, half-planes, rotating calipers and more). Every one of them:

- compiles on its own as strict C++17 with zero warnings (`-Wall -Wextra -Wshadow -Wpedantic`), both as written and under `#define int long long`;
- was checked against a brute-force implementation on hundreds to thousands of random inputs (and planted bugs were confirmed to be caught);
- avoids every name the template defines (`INF`, `MOD`, `sz`, `all`, `rep`, `dbg`, ...) and uses no `__int128`, so it also works on 32-bit judges.

6.1 Three ways to insert a template

1. **Chords.** Ctrl+Alt+T (core), Ctrl+Alt+Y (advanced) or Ctrl+Alt+U (specialist techniques), release, then one key. The code is pasted at the cursor and re-indented to match it. The maps below show every chord.
2. **Prefixes.** Every template has a prefix starting with `cp`: type it and press Tab (`cpdsu`, `cpseg`, `cphld`). Typing just `cp` lists all of them with their descriptions.
3. **Picker.** Ctrl+Alt+T then Space opens VS Code's snippet list; type any part of a name.

6.2 Where to paste

Each template is one of two kinds, shown on its card in Chapter 8 and in `CHEATSHEET.md`:

- **above solve()** (*global*): a `struct` or functions, such as `DSU`, `SegTree`, `dijkstra`. Put the cursor on an empty line above `void solve()` and paste; then use it inside `solve()`.
- **inside solve()**: a block of statements that uses variables you already have, such as the DFS or grid BFS. The first line of the card says which variables it needs (`needs n, adj`). Paste it where those variables exist, then fill in the marked lines.

A few templates build on another one and say so (`Needs cppoint`, `Needs cpdinic`, `Needs cpdsurb`): paste the dependency first.

6.3 Conventions used everywhere

Convention	Meaning
0-indexed	vertices $0..n-1$, array positions $0..n-1$. `cpgraph` converts 1-indexed input for you.
half-open ranges	every range query is on $[l, r)$: `st.query(0, n)` is the whole array, `query(l, l)` is empty.
unreachable	shortest-path templates return `LLONG_MAX` (or the named constant they define) for unreachable vertices.
empty result = impossible	e.g. `topo_sort` returns an empty vector on a cycle, `TwoSat::solve` when unsatisfiable.
`MOD`	modular templates use the template's `MOD`.
edit points	lines to adapt are marked with a comment (`// transition`, `// replace with the real check`).

6.4 Keyboard maps

Orange keys have a template; the short name under the key is its prefix without `cp`. The complete list with descriptions is Chapter 8.

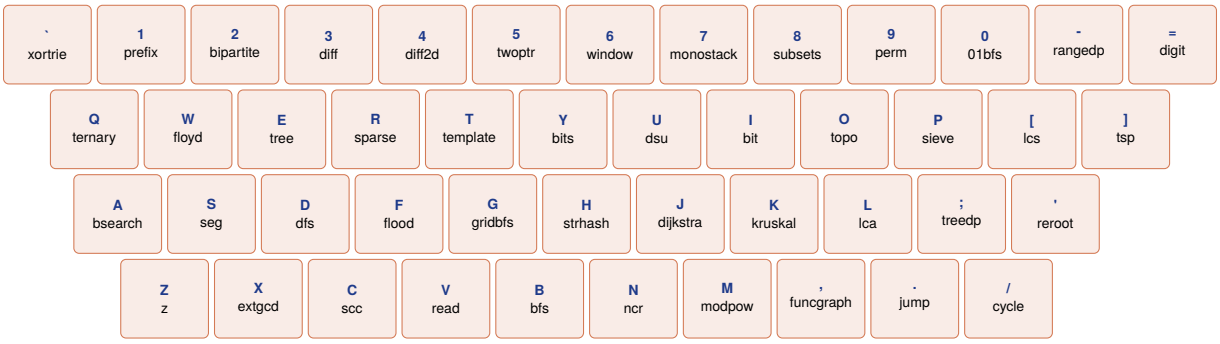


Figure 6.1: Ctrl+Alt+T, then a key no Shift (46 templates)

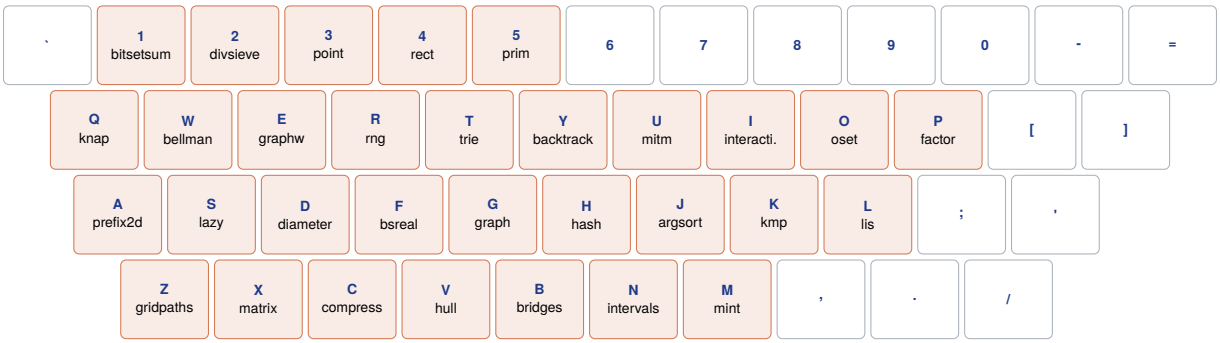


Figure 6.2: Ctrl+Alt+T, then a key with Shift (31 templates)

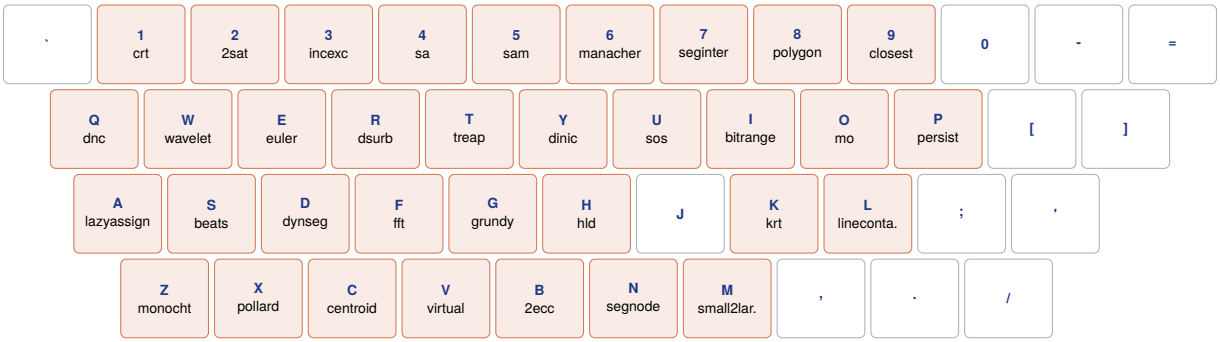


Figure 6.3: Ctrl+Alt+Y, then a key no Shift (34 templates)

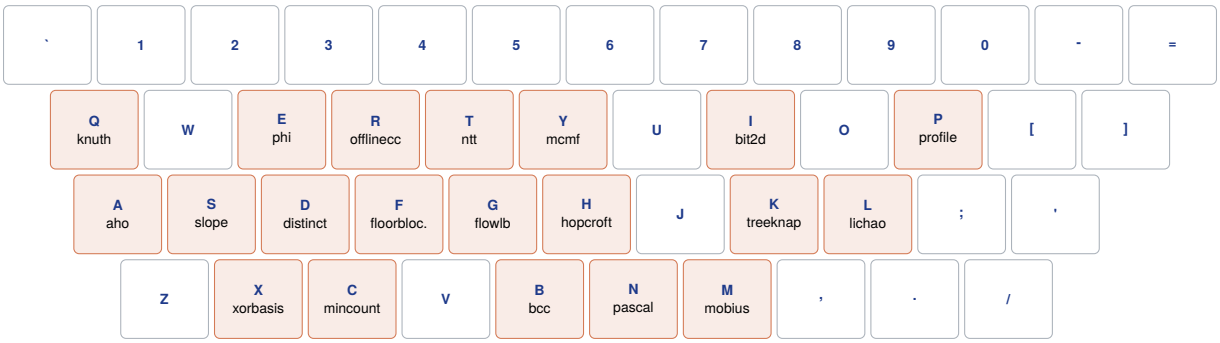


Figure 6.4: Ctrl+Alt+Y, then a key with Shift (20 templates)

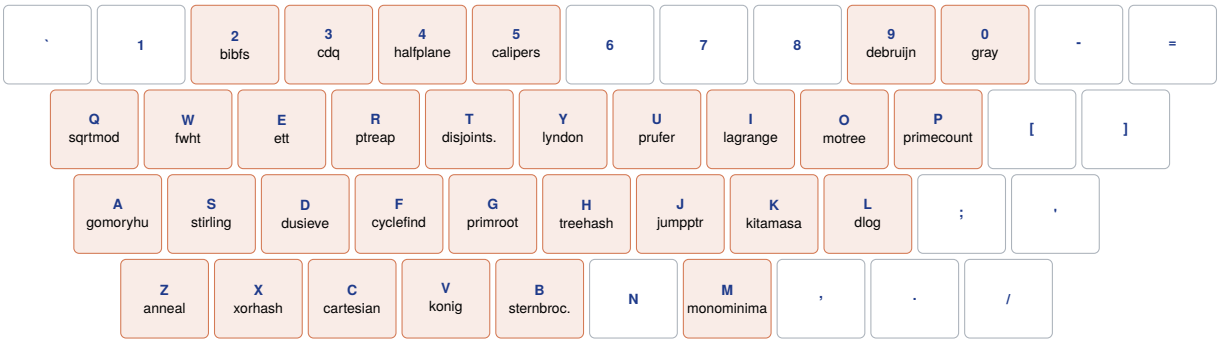


Figure 6.5: Ctrl+Alt+U, then a key no Shift (31 templates)

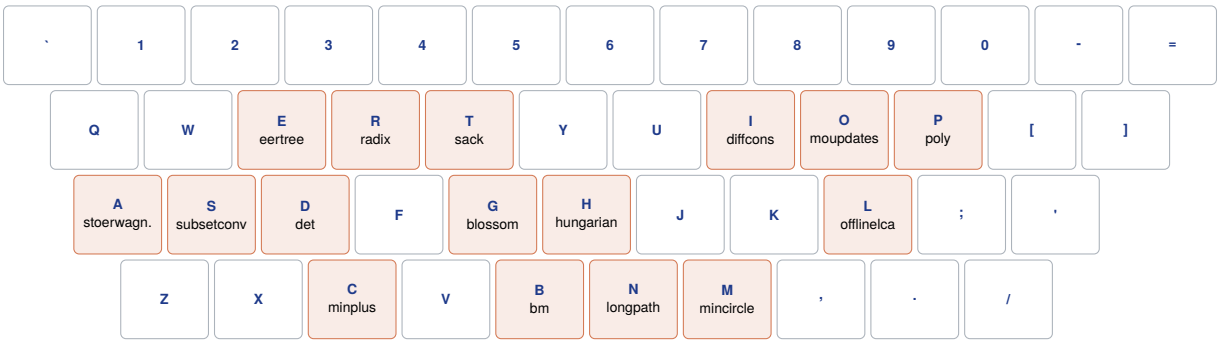


Figure 6.6: Ctrl+Alt+U, then a key with Shift (16 templates)

6.5 Categories

Folder	Contents
basics/	input readers, prefix sums, difference arrays, two pointers, sliding window, monotonic stack, binary and ternary search, subsets, permutations, backtracking, argsort, intervals, bit helpers, meet in the middle, coordinate compression
graph/	DFS, BFS, flood fill, grid BFS, Dijkstra, 0-1 BFS, Bellman–Ford, Floyd–Warshall, topological sort, DSU (+ rollback), Kruskal, Prim, SCC, 2-SAT, bipartite check, bridges and cut vertices, 2-edge-connected components, biconnected components, Euler paths, cycle finding, functional graphs, k-th successor, offline connectivity
tree/	tree DFS / Euler tour, diameter, LCA, tree DP, rerooting, small-to-large, HLD, centroid decomposition, virtual tree, Kruskal reconstruction tree, tree knapsack
ds/	segment trees (point, lazy add, lazy assign, custom node with binary search, min + count, dynamic, persistent, beats), Fenwick trees (point, range, 2D), sparse table, ordered set, anti-hack hash, wavelet tree, treap, Mo's algorithm, offline distinct count
dp/	knapsacks, bitset subset sum, grid paths, LIS, LCS / edit distance, bitmask (TSP), range DP, digit DP, SOS DP, divide-and-conquer and Knuth optimisations, broken profile, LineContainer, Li Chao tree, slope trick
math/	modular power, Mint, nCr, sieve, factorization, Pollard rho, divisor sieve, totient, Möbius, floor blocks, Pascal, extended gcd, CRT, matrix power, FFT, NTT, XOR basis, Grundy numbers, inclusion–exclusion
string/	hashing, KMP, Z-function, trie, binary trie, Aho–Corasick, suffix array + LCP, suffix automaton, Manacher
geometry/	point primitives, segment intersection, polygons (area, containment, Pick), convex hull, closest pair, rectangles
flow/	Dinic + min cut, min-cost flow, flows with lower bounds, Hopcroft–Karp
misc/	random numbers, interactive-problem helpers

CHAPTER 7

Maintenance and Troubleshooting

7.1 VS Code tasks

All tools are VS Code tasks (Ctrl+Shift+P, *Tasks: Run Task*), defined in `.vscode/tasks.json`. Each runs `tools/cp.py` with Python.

Task	Shortcut	Command
`cf-run`	F5	`cp.py test FILE`
CP: Run With Typed Input	Ctrl+F5	`cp.py run FILE`
CP: Stress Test	Ctrl+Alt+X	`cp.py stress FILE`
CP: Quickstart	Ctrl+Shift+N	`cp.py new NAME`
CP: Quickstart USACO	Ctrl+Shift+U	`cp.py new NAME --usaco`
CP: Archive Old Files	Ctrl+Shift+Alt+C	`cp.py archive`
CP: Setup	—	`cp.py setup --check`
CP: Build Docs	—	`cp.py docs`

7.2 CP: Setup

Run it after changing anything in `library/`, the template, or the flags. It is idempotent, and does all of this:

1. rebuilds the precompiled header for the current compiler and flags;
2. regenerates `snippets/cp-library.code-snippets` in the *competitive* profile;
3. regenerates the shortcut block in the profile's `keybindings.json`;
4. writes the four CPH settings (flags, template path, template variables, stderr handling);
5. regenerates `CHEATSHEET.md` and the generated parts of this guide (reference cards, keyboard maps);
6. with `--check`: compiles every template as strict C++17, both as written and under `#define int long long`, and reports any warning.

Reload VS Code once afterwards (Ctrl+Shift+P, *Reload Window*) so new shortcuts take effect.

7.3 Adding your own template

Create a `.cpp` file in the right category folder. The header comments describe it; the code between `// @begin` and `// @end` is what gets pasted. Anything outside the markers exists only so the file

compiles on its own (includes, a wrapper function providing the variables an *inside solve()* snippet uses).

```
// @name   Sum of a range (example)
// @prefix cprangesum
// @key    y z
// @desc   [global] range_sum(P, l, r) = sum of a[l, r) from prefix sums P.
// @usage  ll s = range_sum(P, l, r);
// @complexity O(1)
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// @begin
ll range_sum(const vector<ll>& P, int l, int r) { return P[r] - P[l]; }
// @end
```

- `@key` is `t`, `y` or `u` (the leader) followed by a VS Code key name: a letter, a digit, `shift+x`, or one of `- = [ ] ; ' , . / ``. Setup refuses duplicate keys, prefixes and names.
- `@desc` starts with `[global]` or `[inside solve]`; the rest is the one-line description.
- `@usage` and `@complexity` are optional and feed the reference cards.
- Do not end a `//` comment with a backslash (C++ would glue the next line into the comment).

Then run *CP: Setup* (with its check) and reload VS Code.

7.4 Rebuilding these PDFs

CP: Build Docs (`python tools/cp.py docs`) regenerates the reference and compiles `docs/guide.tex` and `docs/algorithms.tex` with TinyTeX's `pdflatex` (three passes, for the table of contents), writing `docs/GUIDE.pdf` and `docs/ALGORITHMS.pdf`. Build files go to `docs/build/`. `--only guide` or `--only algorithms` builds one of them. The shared look lives in `docs/cpstyle.sty`.

7.5 Troubleshooting

A shortcut does nothing

Reload the window. Check you are in the *competitive* profile (bottom-left gear) and in a `.cpp` file: the template shortcuts only work in C++ editors.

CPH says "Compilation error" about `debug.h`

CPH's flags are missing `-I.../include`. Run *CP: Setup*.

CPH marks a correct test as failed

If its Standard Error box shows output, `ignoreSTDERROR` was turned off; run *CP: Setup*. Otherwise the program exited with a non-zero code: press F5 to see why.

Compiles are slow again

The compiler or flags changed and the precompiled header is stale. Run *CP: Setup* (F5 rebuilds it automatically too).

GCC "internal compiler error" mentioning `bits/stdc++.h`
CPH's flags were edited by hand and no longer match the precompiled header. Run *CP: Setup*.

Red squiggles under `bits/stdc++.h`
IntelliSense is configured in `.vscode/settings.json` to use MSYS2's `g++` in C++17 mode with `LOCAL` defined; if you moved the compiler, update `C_Cpp.default.compilerPath`.

A deep recursion crashes locally
It should not with a 256 MB stack; if it does, the recursion is deeper than about 10^6 levels or each level holds a big local array. Make it iterative or move the array out of the function.

The stress tester says brute.cpp fails a sample
Fix the brute force first; a wrong brute force makes every verdict meaningless.

I changed the template and new files look old
CPH reads `template/template.cpp` at file creation time; existing files are never changed.

7.6 Undoing everything

1. In the profile folder, restore `keybindings.json.bak-cp-setup` and `settings.json.bak-cp-setup` over the current files, and delete `snippets/cp-library.code-snippets`.
2. Restore `template/template.cpp.bak` over `template/template.cpp`.
3. Delete `.vscode/`, `tools/`, `include/`, `library/`, `stress/`, `docs/` and `CHEATSHEET.md` from the folder.

Your solutions and CPH's test cases are never touched by any of the tools except the archive shortcut, which only moves them.

CHAPTER 8

Library Reference

Every template in `library/`, generated from the files themselves by `python tools/cp.py setup`, so this chapter is always in sync with what the shortcuts paste. Each card shows the shortcut, the snippet prefix, where the code belongs, its complexity and the source file, followed by a description and typical usage. The algorithms behind each template are explained in the companion handbook, *ALGORITHMS.pdf*.

8.1 Template

Full template

Ctrl+Alt+T, T · cptemplate · empty file · – · template/template.cpp

the whole template, for files CPH didn't create

8.2 Basics: input, search, sorting, prefix sums, two pointers

2D prefix sums

Ctrl+Alt+T, Shift+A · cpprefix2d · inside solve() · O(nm) build, O(1) per query · library/basics/prefix_sum_2d.cpp

needs n, m, a (n x m). rect(r1, c1, r2, c2) = sum of rows [r1, r2) x cols [c1, c2).

```
ll s = rect(r1, c1, r2, c2);  // rows [r1, r2), cols [c1, c2)
```

All permutations (next_permutation)

Ctrl+Alt+T, 9 · cpperm · inside solve() · O(n! n) · library/basics/permutations.cpp

needs n (n <= ~10). Visits every order of 0..n-1 in lexicographic order.

```
// on a sorted vector v directly: do { ... } while (next_permutation(v.begin(), v.end()
    ));  (skips duplicates)
```

All subsets (bitmask loop)

Ctrl+Alt+T, 8 · cpsubsets · inside solve() · O(2^n n) · library/basics/subsets.cpp

needs n, a (n <= ~20). Visits every subset; bit i of mask set = item i taken.

```
// for all submasks of a mask: for (int s = mask; s; s = (s - 1) & mask)
```

Argsort + custom comparator

Ctrl+Alt+T, Shift+J · cpargsort · above solve() · O(n log n) · library/basics/argsort.cpp

argsort(a): indices sorted by a[i] (stable). Custom orders: sort with a lambda (see usage).

```
vector<int> idx = argsort(a);  // a[idx[0]] is the smallest
```

```
sort(v.begin(), v.end(), [](const auto& x, const auto& y) { return make_pair(x.first, -
    x.second) < make_pair(y.first, -y.second); });
```

Binary search on a real answer

Ctrl+Alt+T, Shift+F cpbsreal inside solve() 100 calls to ok library/basics/binary_search_real.cpp

smallest real x in [lo, hi] with ok(x); ok must be monotone. 100 halvings reach full double precision.

```
// fill in ok(x); lo ends within 1e-15 of the boundary (relative to hi - lo)
```

Binary search on the answer

Ctrl+Alt+T, A cpbsearch inside solve() O(log(hi - lo)) calls to ok library/basics/binary_search.cpp

smallest x in [lo, hi] with ok(x); ok must be monotone (false...false true...true).

```
// fill in ok(x); lo ends as the smallest x with ok(x)
```

Bit helpers

Ctrl+Alt+T, Y cpbits above solve() O(1) library/basics/bits.cpp

popcnt, highest_bit (floor log2), lowest_bit, has_bit for 64-bit values; submask loop in usage.

```
popcnt(x); highest_bit(x); lowest_bit(x); has_bit(x, i);
for (ll s = mask; s; s = (s - 1) & mask) { /* every non-empty submask */ }
```

Coordinate compression

Ctrl+Alt+T, Shift+C cpcompress above solve() O(n log n) library/basics/compress.cpp

auto vals = compress(a); a[i] becomes its rank 0..k-1 and vals[rank] is the original value.

```
vector<int> vals = compress(a);  // a[i] -> rank, vals[rank] -> value
```

Difference array (offline range add, 1D)

Ctrl+Alt+T, 3 cpdiff inside solve() O(1) per update, O(n) build library/basics/difference_array.cpp

needs n. range_add(l, r, v) adds v to a[l, r) in O(1); build() returns the final array.

```
range_add(l, r, v);          // any number of times
vector<ll> a = build();      // once, at the end
```

Difference array (offline rectangle add, 2D)

Ctrl+Alt+T, 4 cpdiff2d inside solve() O(1) per update, O(nm) build library/basics/difference_array_2d.cpp

needs n, m. rect_add(r1, c1, r2, c2, v) adds v to rows [r1, r2) x cols [c1, c2); build() returns the grid.

```
rect_add(r1, c1, r2, c2, v);   // any number of times
auto g = build();              // g[r][c] = total added to cell (r, c)
```

Gray code

Ctrl+Alt+U, 0 cpgray above solve() O(1) / O(log) per value library/basics/gray_code.cpp

gray(i) = i ^ (i >> 1): consecutive values differ in exactly one bit; gray_inverse(g) gives i back. To visit all 2^n subsets changing one element at a time, step k flips bit __builtin_ctzll(k).

```
for (ll k = 1, mask = 0; k < (1LL << n); k++) { int b = __builtin_ctzll(k); mask ^= 1
    LL << b; /* add or remove b */ }
ll g = gray(i), back = gray_inverse(g);
```

Intervals: merge + max disjoint (greedy)

Ctrl+Alt+T, Shift+N cpintervals above solve() O(n log n) library/basics/intervals.cpp

merge_intervals(v): union of closed [l, r] intervals; max_disjoint(v): most pairwise non-overlapping (earliest end first).

```
vector<pair<ll, ll>> segs = merge_intervals(v);
int k = max_disjoint(v);  // touching intervals ([1, 2] and [2, 3]) count as
    overlapping
```

Meet in the middle (subset sums)

Ctrl+Alt+T, Shift+U cpmitm above solve() O(2^(n/2) n) library/basics/meet_in_middle.cpp

count_subsets_leq(a, X): number of subsets with sum <= X for n <= ~40, splitting the items in two halves.

```
ll ways = count_subsets_leq(a, X);
// exact sum X: count upper_bound - lower_bound of X - x in the sorted right half
```

Monotonic stack (next greater / previous smaller)

Ctrl+Alt+T, 7 cpmonostack above solve() O(n) library/basics/monotonic_stack.cpp

next_greater(a)[i] = first j > i with a[j] > a[i] (n if none); prev_smaller(a)[i] = last j < i with a[j] < a[i] (-1).

```
auto nxt = next_greater(a); auto prv = prev_smaller(a);
// largest rectangle in a histogram: width of bar i = next smaller - prev smaller - 1
```

Prefix sums (1D)

Ctrl+Alt+T, 1 cpprefix inside solve() O(n) build, O(1) per query library/basics/prefix_sum.cpp

needs n, a. P[i] = a[0] + ... + a[i-1]; range_sum(l, r) = a[l] + ... + a[r-1].

```
ll s = range_sum(l, r);  // [l, r)
// subarrays with sum k: count pairs i < j with P[j] - P[i] == k using a map of seen P
    values
```

Radix sort and counting sort

Ctrl+Alt+U, Shift+R cpradix above solve() O(n) per pass: O(4 (n + 65536)) for radix_sort, O(n + K) for counting_order library/basics/radix_sort.cpp

radix_sort(a): sorts 64-bit integers (negatives too) in 4 linear passes of 16 bits, often 2-3x faster than std::sort for n >= 1e6; counting_order(keys, K): stable order of indices by small keys in [0, K).

```
radix_sort(a);   // vector<ll>
vector<int> ord = counting_order(keys, K);   // stable: equal keys keep their order
```

Read graph (n, m, m edges)

Ctrl+Alt+T, Shift+G cpgraph inside solve() O(n + m) library/basics/read_graph.cpp

reads n, m and m 1-indexed edges into a 0-indexed undirected adjacency list.

```
// then: dfs / bfs / ... over adj
```

Read vector

Ctrl+Alt+T, V cpread inside solve() O(n) library/basics/read_vector.cpp

needs n. Reads n numbers into a.

```
// with the template: vi a(n); cin >> a;  does the same
```

Read weighted graph (n, m, edges u v w)

Ctrl+Alt+T, Shift+E cpgraphw inside solve() O(n + m) library/basics/read_weighted_graph.cpp

reads n, m and m 1-indexed weighted edges into adj[u] = {v, w}, 0-indexed, undirected.

```
// then: auto [dist, par] = dijkstra(adj, 0);
```

Recursive complete search (backtracking)

Ctrl+Alt+T, Shift+Y cpbacktrack inside solve() O(2^n) calls without pruning library/basics/backtracking.cpp

needs n. Builds every subsequence of 0..n-1 in cur (skip / take); prune at the top of search().

```
// permutations instead: loop over unused items, mark used, recurse, unmark
```

Sliding window min / max (monotonic deque)

Ctrl+Alt+T, 6 cpwindow above solve() O(n) library/basics/sliding_window.cpp

window_min(a, k)[i] = min of a[i, i + k); pass want_max = true for the max. Monotonic deque.

```
vector<ll> mins = window_min(a, k);
vector<ll> maxs = window_min(a, k, true);
```

Ternary search (unimodal max)

Ctrl+Alt+T, Q cpternary above solve() O(log range) calls to f library/basics/ternary_search.cpp

argmax_int(lo, hi, f) / argmax_real(lo, hi, f) for f that rises then falls. For a minimum pass -f.

```
ll x = argmax_int(0, 1e9, [&](ll x) { return profit(x); });
ld y = argmax_real(0, 1e4, [&](ld x) { return -cost(x); });  // minimum of cost
```

Two pointers (sliding window)

Ctrl+Alt+T, 5 cptwoptr inside solve() O(n) library/basics/two_pointers.cpp

needs n, a (values >= 0). Longest window with sum <= S; grow r, shrink l while the window is invalid.

```
// count windows instead: add (r - l + 1) to an answer inside the loop
```

8.3 Graphs

0-1 BFS

Ctrl+Alt+T, 0 cp01bfs above solve() O(n + m) library/graph/zero_one_bfs.cpp

vector<ll> dist = zero_one_bfs(adj, src); adj[u] = {v, w} with w in {0, 1}. Unreachable = LLONG_MAX.

```
vector<ll> dist = zero_one_bfs(adj, src);  // adj[u] = {v, w}, w in {0, 1}
```

2-edge-connected components

Ctrl+Alt+Y, B cp2ecc above solve() O(n + m) library/graph/two_edge_cc.cpp

auto [k, comp] = two_edge_cc(n, edges): vertices stay connected after deleting any single edge iff same comp. Bridges join comps.

```
auto [k, comp] = two_edge_cc(n, edges);
// bridge tree: for each edge (u, v) with comp[u] != comp[v], connect comp[u] - comp[v
    ]
```

2-SAT

Ctrl+Alt+Y, 2 cp2sat above solve() O(n + clauses) library/graph/two_sat.cpp

TwoSat ts(n); ts.either(a, va, b, vb) adds the clause (x_a == va OR x_b == vb); ts.solve() -> values or empty if impossible.

```
TwoSat ts(n);
ts.either(a, true, b, false); ts.implies(a, true, c, true); ts.set(d, false);
vector<int> x = ts.solve();  // empty: unsatisfiable
```

Bellman-Ford (negative edges)

Ctrl+Alt+T, Shift+W cpbellman above solve() O(nm) library/graph/bellman_ford.cpp

bellman_ford(n, edges, src), edges = {u, v, w}. BF_INF = unreachable, -BF_INF = hit by a negative cycle.

```
vector<array<ll, 3>> edges;  // {u, v, w}
vector<ll> dist = bellman_ford(n, edges, src);  // -BF_INF: negative cycle reaches it
```

BFS (shortest path, unweighted)

Ctrl+Alt+T, B cpbfs inside solve() O(n + m) library/graph/bfs.cpp

needs n, adj, src. dist[v] = -1 if unreachable; get_path(t) rebuilds the route.

```
// after it: dist[v] (-1 = unreachable), par[v]
vector<int> path = get_path(t);  // src ... t
```

Biconnected components (blocks)

Ctrl+Alt+Y, Shift+B cpbcc above solve() O(n + m) library/graph/biconnected_components.cpp

biconnected_components(n, edges): every block (maximal subgraph with no cut vertex) as a vertex list. Cut vertices appear in several.

```
auto blocks = biconnected_components(n, edges);
// block-cut tree: node per block + node per cut vertex, edge when the cut vertex is
    in the block
```

Bidirectional BFS (meet in the middle on shortest paths)

Ctrl+Alt+U, 2 cpbibfs above solve() O(b^(d/2)) states for branching factor b and distance d, instead of O(b^d) library/graph/bidirectional_bfs.cpp

bidirectional_bfs(s, t, neighbours): unweighted distance from s to t (-1 if unreachable) exploring from both ends; neighbours(state) returns a vector of states. Works on implicit graphs (puzzles, strings) with any hashable State.

```
ll d = bidirectional_bfs<ll>(start, goal, [&](ll s) { vector<ll> nb; /* moves */
    return nb; });
```

Bipartite check (2-coloring)

Ctrl+Alt+T, 2 cpbipartite above solve() O(n + m) library/graph/bipartite.cpp

vector<int> color = bipartite(adj); colors are 0/1, empty if an odd cycle exists.

```
vector<int> color = bipartite(adj);
if (color.empty()) { /* odd cycle: not bipartite */ }
```

Bridges + articulation points

Ctrl+Alt+T, Shift+B cpbridges above solve() O(n + m) library/graph/bridges.cpp

auto [bridge_ids, cut_vertices] = bridges_and_cuts(n, edges); undirected, multi-edges OK.

```
vector<pair<int, int>> edges;  // undirected {u, v}
auto [bridge_ids, cut_vertices] = bridges_and_cuts(n, edges);
```

Cycle finding in O(1) memory (Floyd / Brent)

Ctrl+Alt+U, F cpcyclefind above solve() O(mu + lambda) evaluations of f, O(1) memory library/graph/cycle_floyd.cpp

find_rho(x0, f): for the sequence x0, f(x0), f(f(x0)), ... returns {mu, lambda}: the first index of the cycle and its length. Brent's variant: O(mu + lambda) calls of f, O(1) memory (use when states are huge).

```
auto [mu, lam] = find_rho<ll>(seed, [](ll x) { return (x * x + 1) % 1'000'003; });
```

DFS (recursive, all components)

Ctrl+Alt+T, D cpdfs inside solve() O(n + m) library/graph/dfs.cpp

needs n, adj. Labels every vertex with its component id.

```
// after it: comp[v] = component of v, ncomp = number of components
```

Dijkstra

Ctrl+Alt+T, J cpdijkstra above solve() O((n + m) log n) library/graph/dijkstra.cpp

auto [dist, par] = dijkstra(adj, src); adj[u] = {v, w}, w >= 0. Unreachable = LLONG_MAX.

```
vector<vector<pair<int, ll>>> adj(n);  // adj[u].push_back({v, w})
auto [dist, par] = dijkstra(adj, src);
```

DSU (union-find)

Ctrl+Alt+T, U cpdsu above solve() ~O(1) amortized per call library/graph/dsu.cpp

DSU d(n); d.unite(a, b) -> false if already joined; d.same, d.size, d.comps.

```
DSU d(n);
if (d.unite(a, b)) { /* merged two components */ }
d.same(a, b); d.size(a); d.comps;
```

DSU with rollback

Ctrl+Alt+Y, R cpdsurb above solve() O(log n) per find / unite, O(1) per undone unite library/graph/dsu_rollback.cpp

DSURollback d(n); d.unite(a, b); int t = d.snapshot(); ... d.rollback(t) undoes every unite after t. No path compression.

```
DSURollback d(n);
int t = d.snapshot(); d.unite(a, b); /* ... */ d.rollback(t);
```

Eulerian path / circuit (Hierholzer)

Ctrl+Alt+Y, E cpeuler above solve() O(n + m) library/graph/euler_path.cpp

euler_path(n, edges, directed, src = -1): vertex sequence using every edge exactly once, or empty if impossible.

```
vector<int> path = euler_path(n, edges, false);       // path.size() == m + 1
vector<int> tour = euler_path(n, edges, false, 0);    // circuit from 0 (only when
    every degree is even)
```

Find a directed cycle

Ctrl+Alt+T, / cpcycle above solve() O(n + m) library/graph/find_cycle.cpp

find_cycle(adj): vertices of some directed cycle in order (the last one has an edge back to the first); empty if acyclic.

```
vector<int> cyc = find_cycle(adj);
// undirected graphs: a DSU finds the first edge whose ends are already joined
```

Flood fill (grid regions)

Ctrl+Alt+T, F cpflood inside solve() O(nm) library/graph/flood_fill.cpp

needs n, m, grid (vector<string>). Iterative, so big grids can't overflow the stack.

```
// after it: region[r][c] = region id (-1 = not open), region_size[id]
// change open(r, c) to choose which cells get filled
```

Floyd-Warshall (all pairs)

Ctrl+Alt+T, W cpfloyd above solve() O(n^3) library/graph/floyd_warshall.cpp

floyd_warshall(d) in place, O(n^3). d[i][i] < 0 afterwards <=> i is on a negative cycle.

```
vector<vector<ll>> d(n, vector<ll>(n, FW_INF));
for (int i = 0; i < n; i++) d[i][i] = 0;  // then d[u][v] = min(d[u][v], w) per edge
floyd_warshall(d);
```

Functional graph (successor graph) cycles

Ctrl+Alt+T, , cpfuncgraph above solve() O(n) library/graph/functional_graph.cpp

FunctionalGraph fg(f), f[v] = the one successor of v: fg.cyc[v] (cycle id or -1), fg.cyc_len[id], fg.dist[v] to its cycle, fg.entry[v].

```
FunctionalGraph fg(f);
int steps_until_repeat = fg.dist[v] + fg.cyc_len[fg.cyc[fg.entry[v]]];
```

Grid BFS (shortest path on a grid)

Ctrl+Alt+T, G cpgridbfs inside solve() O(nm) library/graph/grid_bfs.cpp

needs n, m, grid. Starts from every 'S' (multi-source), '#' is a wall.

```
// after it: dist[r][c] = steps from the nearest 'S' (-1 = unreachable)
```

K-th successor (binary lifting / jumping)

Ctrl+Alt+T, . cpjump above solve() O(n log K) build, O(log K) per query library/graph/kth_successor.cpp

Jump J(f, max_k); J.go(v, k) = f applied k times to v, for k <= max_k. Works for any functional graph.

```
Jump J(f, 1e9);
int where = J.go(v, k);
```

Kruskal MST

Ctrl+Alt+T, K cpkruskal above solve() O(m log m) library/graph/kruskal.cpp

auto [cost, used] = kruskal(n, edges); edges = {u, v, w}. Spanning forest if disconnected.

```
vector<array<ll, 3>> edges;  // {u, v, w}
auto [cost, used] = kruskal(n, edges);
```

Offline dynamic connectivity (edge deletions)

Ctrl+Alt+Y, Shift+R cpofflinecc above solve() O(q log q log n) library/graph/offline_connectivity.cpp

offline_components(n, ops): ops are {0, u, v} add edge, {1, u, v} remove it, {2, 0, 0} query; returns #components per query. Needs cpdsurb.

```
vector<array<int, 3>> ops;  // {type, u, v}
vector<int> answers = offline_components(n, ops);
```

Prim MST (dense graphs)

Ctrl+Alt+T, Shift+5 cpprim above solve() O(n^2) library/graph/prim.cpp

prim_dense(w): MST weight for an n x n weight matrix (complete graphs, e.g. points in the plane). O(n^2).

```
vector<vector<ll>> w(n, vector<ll>(n));  // w[i][j] = cost of edge i-j
ll cost = prim_dense(w);
```

SCC (Tarjan)

Ctrl+Alt+T, C cpscc above solve() O(n + m) library/graph/scc.cpp

auto [k, comp] = scc(adj). Ids are reverse-topological: edge u->v across SCCs has comp[u] > comp[v].

```
auto [k, comp] = scc(adj);  // k components, comp[v] in [0, k)
```

System of difference constraints

Ctrl+Alt+U, Shift+I cpdiffcons above solve() O(n m) (Bellman-Ford from a virtual source) library/graph/difference_constraints.cpp

DiffConstraints dc(n); dc.add(u, v, w) means x_v - x_u <= w; dc.solve() -> a solution vector (all x <= 0, maximal), or empty if infeasible (a negative cycle).

```
DiffConstraints dc(n); dc.add(a, b, 3);   // x_b <= x_a + 3
vector<ll> x = dc.solve(); if (x.empty()) { /* impossible */ }
```

Topological sort (Kahn)

Ctrl+Alt+T, O cptopo above solve() O(n + m) library/graph/topo_sort.cpp

vector<int> order = topo_sort(adj); empty if the graph has a cycle.

```
vector<int> order = topo_sort(adj);
if (order.empty()) { /* the graph has a cycle */ }
```

8.4 Trees

Centroid decomposition

Ctrl+Alt+Y, C cpcentroid above solve() O(n log n) library/tree/centroid.cpp

CentroidDecomp cd(adj): cd.cpar[v] = parent in the centroid tree (-1 at the top), cd.clevel[v] = its depth (<= log2 n).

```
CentroidDecomp cd(adj);
for (int c = v; c != -1; c = cd.cpar[c]) { /* every path through v passes a centroid
    ancestor c: use dist(v, c) */ }
```

DSU on tree (Sack): subtree statistics in one shared array

Ctrl+Alt+U, Shift+T cpsack inside solve() O(n log n) calls of add library/tree/sack.cpp

needs n, adj, root, color (values in [0, n)). Keeps the heavy child's counts and re-adds the light subtrees: best[v] = the largest count of one color in v's subtree. Edit add / clear / the answer line.

```
// add(v) puts one vertex into cnt[] and the statistic; clear(v) empties the subtree
    of v again
```

Heavy-light decomposition

Ctrl+Alt+Y, H cphld above solve() O(n) build, O(log n) ranges per path library/tree/hld.cpp

HLD h(adj, root); h.for_path(a, b, f) calls f(l, r) on position ranges [l, r) covering the path; pair with any [l, r) segment tree.

```
HLD h(adj); SegTree st(n);                    // st.set(h.pos[v], value of v)
ll s = 0; h.for_path(a, b, [&](int l, int r) { s += st.query(l, r); });
// subtree of v = [h.pos[v], h.pos[v] + h.sub[v]); h.lca(a, b)
```

Jump pointers (level ancestors with one extra pointer per vertex)

Ctrl+Alt+U, J cpjumpptr above solve() O(1) per added leaf, O(log n) per query library/tree/jump_pointers.cpp

JumpPointers jp(n); jp.add_root(r); jp.add_leaf(v, p) as vertices arrive (parent first); jp.kth(v, k), jp.ancestor_at_depth(v, d), jp.lca(a, b). Works online while the tree grows.

```
JumpPointers jp(n); jp.add_root(0);
for (int v : order) if (v != 0) jp.add_leaf(v, par[v]);  // any order with parents
    before children
int a = jp.kth(v, k);  // -1 if v has fewer than k ancestors
```

Kruskal reconstruction tree

Ctrl+Alt+Y, K cpkrt above solve() O(m log m) library/tree/kruskal_reconstruction.cpp

KRT t(n, edges {u, v, w}): leaves 0..n-1, each merge makes node x with val[x] = w. Max edge on the MST path a-b = val[lca(a, b)].

```
KRT t(n, edges); auto g = t.tree();  // then LCA l(g, g.size() - 1) if the graph is
    connected
// vertices reachable from v using edges <= x = leaves under v's highest ancestor with
     val <= x
```

LCA (binary lifting)

Ctrl+Alt+T, L cplca above solve() O(n log n) build, O(log n) per query library/tree/lca.cpp

LCA t(adj, root); t.lca(a, b), t.dist(a, b), t.kth_ancestor(v, k), t.depth[v].

```
LCA t(adj, root);
t.lca(a, b); t.dist(a, b); t.kth_ancestor(v, k); t.depth[v];
```

Long-path decomposition: depth-indexed tree DP in O(n)

Ctrl+Alt+U, Shift+N cplongpath inside solve() O(n) library/tree/long_path.cpp

needs n, adj, root. f_v[d] = # vertices at distance d below v lives at buf[pos[v] + d], shared with the long child (shifted by one); dom[v] = the smallest d maximising f_v[d]. Edit the merge for other depth DPs.

```
// the array of v has h[v] cells: buf[pos[v]] .. buf[pos[v] + h[v] - 1]
```

Offline LCA (Tarjan)

Ctrl+Alt+U, Shift+L cpofflinelca above solve() O((n + q) alpha(n)) library/tree/offline_lca.cpp

tarjan_lca(adj, root, qs): the LCA of every query pair, answered in one DFS with a DSU. Faster than binary lifting when all queries are known in advance and memory is tight.

```
vector<int> ans = tarjan_lca(adj, 0, qs);   // qs: vector<pair<int, int>>
```

Prufer code (tree <-> sequence)

Ctrl+Alt+U, U cpprufer above solve() O(n) (linear pointer technique) library/tree/prufer.cpp

prufer_encode(n, edges): the Prufer sequence (length n-2) of a labelled tree on 0..n-1; prufer_decode(seq): the tree back. Bijection behind Cayley's formula n^(n-2); vertex v appears deg(v)-1 times.

```
vector<int> seq = prufer_encode(n, edges);
vector<pair<int, int>> tree = prufer_decode(seq);
```

Rerooting (DP for every root)

Ctrl+Alt+T, ' cpreroot above solve() O(n) merges and lifts library/tree/rerooting.cpp

reroot<T>(adj, id, merge, lift): res[v] = merge over v's neighbours w of lift(value of w's side, w). merge associative, id its identity.

```
// farthest vertex from every v:  auto far = reroot<ll>(adj, 0, [](ll a, ll b) {
    return max(a, b); }, [](ll x, int) { return x + 1; });
// sum of distances: T = pair<ll, ll> {sum, count}, lift(p, w) = {p.first + p.second +
    1, p.second + 1}, merge = add
```

Small-to-large merging

Ctrl+Alt+Y, M cpsmall2large inside solve() O(n log^2 n) library/tree/small_to_large.cpp

needs n, adj, root, color. distinct[v] = # distinct colors in v's subtree; always pour the smaller set into the larger.

```
// swap set<int> for map<int, int> to keep counts (e.g. most frequent color per
    subtree)
```

Tree DFS (parent, depth, subtree size, Euler tour)

Ctrl+Alt+T, E cptree inside solve() O(n) library/tree/tree_dfs.cpp

needs n, adj, root. Subtree of u = Euler positions [tin[u], tout[u]).

```
// after it: par, depth, sub (subtree size), tin/tout (Euler tour)
// subtree of u = positions [tin[u], tout[u]) -> a range query on the tour
```

Tree diameter

Ctrl+Alt+T, Shift+D cpdiameter above solve() O(n) library/tree/tree_diameter.cpp

auto [len, path] = tree_diameter(adj); two BFS passes, path lists the vertices end to end.

```
auto [len, path] = tree_diameter(adj);
```

Tree DP (subtree DP, iterative)

Ctrl+Alt+T, ; cptreedp inside solve() O(n) library/tree/tree_dp.cpp

needs n, adj, root. Children before parents (reverse BFS order); example: maximum independent set.

```
// answer: max(dp[root][0], dp[root][1]); change the two lines marked "transition"
```

Tree isomorphism (canonical IDs, AHU)

Ctrl+Alt+U, H cptreehash above solve() O(n log n) library/tree/tree_hash.cpp

TreeCanon tc; tc.rooted(adj, root) and tc.unrooted(adj) give integer IDs: two trees get the same ID (from the same TreeCanon object) iff they are isomorphic. rooted_all(adj, root)[v] is the ID of v's subtree.

```
TreeCanon tc;
bool iso = tc.unrooted(adj1) == tc.unrooted(adj2);
```

Tree knapsack (merging child DPs)

Ctrl+Alt+Y, Shift+K cptreeknap inside solve() O(n^2) library/tree/tree_knapsack.cpp

needs n, adj, root, w. best[v][j] = max value choosing j vertices of v's subtree, each chosen vertex's parent chosen too.

```
// answer for exactly k vertices containing the root: best[root][k]; merging costs O(n
    ^2) in total
```

Virtual (auxiliary) tree

Ctrl+Alt+Y, V cpvirtual above solve() O(k log k) LCA calls library/tree/virtual_tree.cpp

virtual_tree(nodes, tin, lca) -> {vertices, parent}: key nodes + pairwise LCAs, parent[i] = virtual parent of vertices[i] (-1 root).

```
auto [vs, vpar] = virtual_tree(keys, tin, [&](int a, int b) { return t.lca(a, b); });
     // tin from cptree, t from cplca
// then DP over vs in reverse order: children appear after their parents
```

8.5 Data structures & range queries

Anti-hack hash for unordered_map

Ctrl+Alt+T, Shift+H cphash above solve() O(1) expected per operation library/ds/custom_hash.cpp

unordered_map<ll, int, custom_hash> mp; also works for pair keys. Stops anti-hash TLE hacks.

```
unordered_map<ll, int, custom_hash> cnt;
unordered_map<pair<int, int>, int, custom_hash> seen;
```

Cartesian tree

Ctrl+Alt+U, C cpcartesian above solve() O(n) (one stack pass) library/ds/cartesian_tree.cpp

CartesianTree ct(a): min-heap on values, in-order = array order (ties: the leftmost is higher). ct.root, ct.par[i], ct.lc[i], ct.rc[i] (-1 = none). The subtree of i is the maximal range where a[i] is the minimum.

```
CartesianTree ct(a);
int r = ct.root; for (int i = 0; i < n; i++) { int p = ct.par[i]; }
```

CDQ divide and conquer (3D dominance counting)

Ctrl+Alt+U, 3 cpcdq above solve() O(n log^2 n) library/ds/cdq.cpp

cdq_dominance(pts): for each point i, the number of j != i with a_j <= a_i, b_j <= b_i, c_j <= c_i. The pattern (sort by one key, divide and conquer on the second, BIT on the third) solves offline 3D problems.

```
vector<array<ll, 3>> pts(n);   // (a, b, c)
vector<int> dom = cdq_dominance(pts);
```

Disjoint sparse table (O(1) queries for any associative operation)

Ctrl+Alt+U, T cpdisjointsparse above solve() O(n log n) build, O(1) query library/ds/disjoint_sparse_table.cpp

DisjointSparse<T, Op> t(a, op); t.query(l, r) = a[l] op ... op a[r-1] for [l, r), l < r. Works for sums, products mod p, matrix products: no need for idempotence.

```
DisjointSparse t(a, [](ll x, ll y) { return x * y % MOD; });
ll prod = t.query(l, r);
```

Distinct values in ranges (offline sweep + BIT)

Ctrl+Alt+Y, Shift+D cpdistinct above solve() O((n + q) log n) library/ds/distinct_in_range.cpp

distinct_in_ranges(a, queries): number of distinct values in a[l, r) per query. Sweeps r, keeps only last occurrences.

```
vector<int> res = distinct_in_ranges(a, qs);  // qs[i] = {l, r}
```

Dynamic (sparse) segment tree

Ctrl+Alt+Y, D cpdynseg above solve() O(log range) per operation, O(q log range) memory library/ds/dynamic_segtree.cpp

DynSeg st(lo, hi): positions in [lo, hi) as large as 1e18, nodes made on demand; st.add(pos, v), st.sum(l, r) on [l, r).

```
DynSeg st(0, 1e9);
st.add(x, 1); ll c = st.sum(l, r);
```

Euler tour tree (dynamic forest: link, cut, connectivity, component sums)

Ctrl+Alt+U, E cpett above solve() O(log n) expected per operation library/ds/euler_tour_tree.cpp

ETT f(n); f.link(u, v), f.cut(u, v) (the edge must exist), f.connected(u, v), f.set(v, x), f.comp_sum(v), f.comp_size(v). Each tree is stored as its Euler tour in a treap.

```
ETT f(n); f.link(0, 1); f.set(1, 5);
bool c = f.connected(0, 1); ll s = f.comp_sum(0); f.cut(0, 1);
```

Fenwick tree / BIT

Ctrl+Alt+T, I cpbit above solve() O(log n) per operation library/ds/fenwick.cpp

BIT b(n); b.add(i, v), b.sum(l, r) on [l, r), b.prefix(i) = sum of [0, i), b.lower_bound(k).

```
BIT b(n);
b.add(i, v); b.sum(l, r); b.prefix(i); b.lower_bound(k);
```

Fenwick tree 2D

Ctrl+Alt+Y, Shift+I cpbit2d above solve() O(log n log m) per operation library/ds/fenwick_2d.cpp

BIT2D b(n, m); b.add(r, c, v); b.sum(r1, c1, r2, c2) = sum over rows [r1, r2) x cols [c1, c2).

```
BIT2D b(n, m);
b.add(r, c, 1); ll s = b.sum(r1, c1, r2, c2);
```

Fenwick tree: range add + range sum

Ctrl+Alt+Y, I cpbitrange above solve() O(log n) per operation library/ds/fenwick_range.cpp

BITRange b(n); b.add(l, r, v) adds v on [l, r); b.sum(l, r). Two Fenwick trees over the difference array.

```
BITRange b(n);
```

```
b.add(l, r, v); ll s = b.sum(l, r);
```

Implicit treap (sequence with split / merge)

Ctrl+Alt+Y, T cptreap above solve() O(log n) expected per operation library/ds/treap.cpp

Treap tr; tr.insert(pos, v), tr.erase(pos), tr.reverse_range(l, r), tr.range_sum(l, r), tr.to_vector(). Expected O(log n).

```
Treap tr; for (auto x : a) tr.insert(tr.count(), x);
tr.reverse_range(l, r); ll s = tr.range_sum(l, r); vector<ll> now = tr.to_vector();
```

Lazy segment tree (range add; range sum + min)

Ctrl+Alt+T, Shift+S cplazy above solve() O(log n) per operation library/ds/lazy_segment_tree.cpp

LazySeg st(n) or st(a); st.add(l, r, v), st.sum_of(l, r), st.min_of(l, r), all on [l, r).

```
LazySeg st(a);
st.add(l, r, v); st.sum_of(l, r); st.min_of(l, r);  // all [l, r)
```

Lazy segment tree (range assign + range add; sum, min)

Ctrl+Alt+Y, A cplazyassign above solve() O(log n) per operation library/ds/lazy_assign.cpp

LazyAssign st(n or a): st.assign(l, r, v), st.add(l, r, v), st.sum_of(l, r), st.min_of(l, r), all on [l, r).

```
LazyAssign st(a);
st.assign(l, r, 5); st.add(l, r, -2); st.sum_of(l, r); st.min_of(l, r);
```

Mo's algorithm (offline range queries)

Ctrl+Alt+Y, O cpmo above solve() O((n + q) sqrt n) add/remove calls library/ds/mo_algorithm.cpp

mo(n, queries, add, remove, answer): queries [l, r); add(i) / remove(i) update the window, answer(qi) records query qi.

```
vector<int> cnt(maxv), res(q); int distinct = 0;
mo(n, qs, [&](int i) { distinct += cnt[a[i]]++ == 0; }, [&](int i) { distinct -= --cnt
    [a[i]] == 0; }, [&](int qi) { res[qi] = distinct; });
```

Mo's algorithm on tree paths

Ctrl+Alt+U, O cpmotree above solve() O((n + q) sqrt(n) * cost of add / remove + q log n) library/ds/mo_tree.cpp

mo_on_tree(adj, root, qs, add, remove, answer): offline queries on the vertex set of the path u-v. add(v)/remove(v) change the current set, answer(i) records query i. Uses the Euler tour trick (a vertex seen twice cancels).

```
mo_on_tree(adj, 0, qs, [&](int v) { if (cnt[c[v]]++ == 0) distinct++; }, [&](int v) {
    if (--cnt[c[v]] == 0) distinct--; }, [&](int i) { ans[i] = distinct; });
```

Mo's algorithm with updates (time as a third dimension)

Ctrl+Alt+U, Shift+O cpmoupdates above solve() O(n^(5/3)) for n queries and updates library/ds/mo_updates.cpp

mo_with_updates(n, qs, add, remove, apply, answer): qs[i] = {l, r, t} asks about [l, r) after the first t updates. apply(u, L, R) toggles update u (swap in/out) given the window [L, R).

```
mo_with_updates(n, qs, add, remove, [&](int u, int L, int R) { auto& [p, v] = upd[u];
    if (L <= p && p < R) remove(p); swap(a[p], v); if (L <= p && p < R) add(p); },
    [&](int i) { ans[i] = cur; });
```

Ordered set (PBDS order statistics)

Ctrl+Alt+T, Shift+O cposet above solve() O(log n) per operation library/ds/ordered_set.cpp

ordered_set<int> s; s.find_by_order(k) = k-th smallest (0-based), s.order_of_key(x) = # elements < x.

```
ordered_set<int> s;
s.insert(x); *s.find_by_order(k); s.order_of_key(x);
```

Persistent implicit treap (rope with versions)

Ctrl+Alt+U, R cpptreap above solve() O(log n) expected per operation, O(log n) new nodes per update library/ds/persistent_treap.cpp

PTreap t; int v = t.build(a); every operation returns a NEW version and keeps the old: insert(v, pos, x), erase(v, pos), concat(v1, v2), slice(v, l, r). Queries: get(v, pos), sum(v, l, r), size(v).

```
PTreap t; int v0 = t.build(a); int v1 = t.insert(v0, 2, 7);
int copy_paste = t.concat(v1, t.slice(v1, l, r)); ll s = t.sum(v0, l, r);
```

Persistent segment tree (k-th smallest in range)

Ctrl+Alt+Y, P cppersist above solve() O(log n) per operation and O(log n) new nodes per update library/ds/persistent_segtree.cpp

PersistentSeg ps(n): root2 = ps.update(root1, pos, v) keeps every version; ps.query(root, l, r); ps.kth(rootL, rootR, k).

```
vector<int> root{0}; for (int i = 0; i < n; i++) root.push_back(ps.update(root[i],
    rank[i], 1));  // ranks from cpcompress
int idx = ps.kth(root[l], root[r], k);  // k-th smallest (0-based) of a[l, r) is vals[
    idx]
```

Segment tree (point update, range query)

Ctrl+Alt+T, S cpseg above solve() O(log n) per operation library/ds/segment_tree.cpp

SegTree st(n) or st(a); st.set(i, x), st.get(i), st.query(l, r) on [l, r). Edit T / ID / op.

```
SegTree st(a);  // or SegTree st(n)
st.set(i, x); st.get(i); st.query(l, r);  // [l, r)
```

Segment tree beats (range chmin, sum, max)

Ctrl+Alt+Y, S cpbeats above solve() O((n + q) log^2 n) amortized library/ds/segtree_beats.cpp

Beats st(a): st.chmin(l, r, x) sets a[i] = min(a[i], x) on [l, r); st.sum(l, r), st.max(l, r). Amortized O(log^2 n).

```
Beats st(a);
st.chmin(l, r, x); ll s = st.sum(l, r); ll m = st.max(l, r);
```

Segment tree with custom nodes + binary search

Ctrl+Alt+Y, N cpsegnode above solve() O(log n) per operation library/ds/segtree_node.cpp

SegNode st(n or leaves): any associative Node (example: max subarray sum); st.max_right(l, pred) walks down in O(log n).

```
SegNode st(n); st.set(i, Node::leaf(x)); Node r = st.query(l, r); r.best;
int r = st.max_right(l, [&](const Node& x) { return x.sum < k; });  // first r with
    sum of [l, r] >= k is r
```

Segment tree: range add, min + count of min

Ctrl+Alt+Y, Shift+C cpmincount above solve() O(log n) per operation library/ds/min_count_segtree.cpp

MinCount st(weights): add(l, r, v); query(l, r) = {min, total weight at the min}; zero_weight() = weight of cells equal to 0 (values must stay >= 0, as coverage counts do).

```
MinCount st(len);  // len[i] = width of elementary interval i (all 1 for plain
    counting)
st.add(l, r, +1); st.add(l, r, -1); ll covered = total_width - st.zero_weight();  //
    union of rectangles sweep
```

Sparse table (O(1) range min)

Ctrl+Alt+T, R cpsparse above solve() O(n log n) build, O(1) per query library/ds/sparse_table.cpp

SparseTable st(a); st.query(l, r) = min of [l, r), l < r. Swap op for max/gcd (idempotent only).

```
SparseTable st(a);
st.query(l, r);  // min of [l, r)
```

Wavelet tree (k-th smallest / count <= x in range)

Ctrl+Alt+Y, W cpwavelet above solve() O(n log V) build, O(log V) per query (V = value range) library/ds/wavelet_tree.cpp

Wavelet wt(a): wt.kth(l, r, k) = k-th smallest (0-based) of a[l, r); wt.count_leq(l, r, x). Compress values first for speed.

```
Wavelet wt(a);
ll med = wt.kth(l, r, (r - l) / 2); int c = wt.count_leq(l, r, x);
```

8.6 Dynamic programming

(min, +) convolution with convex sequences

Ctrl+Alt+U, Shift+C cpminplus above solve() O(n + m) / O((n + m) log(n + m)) library/dp/minplus_convex.cpp

c[k] = min over i + j = k of a[i] + b[j]. minplus_convex_convex(a, b): both convex, O(n + m) by merging slopes; minplus_convex_any(a, b): a convex, b arbitrary, O((n + m) log).

```
vector<ll> c = minplus_convex_convex(a, b);
vector<ll> d = minplus_convex_any(convex_a, arbitrary_b);
```

Bitmask DP (travelling salesman)

Ctrl+Alt+T,] cptsp above solve() O(2^n n^2) library/dp/bitmask_tsp.cpp

tsp(dist): shortest cycle through all n <= ~18 vertices from 0. dp[mask][v] = best path covering mask, ending at v.

```
ll best = tsp(dist);  // dist[i][j] = cost i -> j
// Hamiltonian path count: same loops with dp[mask | 1 << v][v] += dp[mask][u] over
    edges
```

Broken profile DP (domino tilings)

Ctrl+Alt+Y, Shift+P cpprofile above solve() O(n m 2^m) library/dp/broken_profile.cpp

domino_tilings(n, m): ways to tile an n x m board with 1 x 2 dominoes mod MOD, cell by cell with an m-bit profile (m <= ~12).

```
ll ways = domino_tilings(n, m);  // put the smaller side as m
```

Digit DP

Ctrl+Alt+T, = cpdigit above solve() O(digits * states * 10) library/dp/digit_dp.cpp

count_upto(N) = how many x in [0, N] have no two equal adjacent digits. Change the state / check for your property.

```
ll ans = count_upto(B) - count_upto(A - 1);  // count in [A, B]
```

Divide & conquer DP optimization

Ctrl+Alt+Y, Q cpdnc above solve() O(K n log n) cost calls library/dp/dnc_optimization.cpp

dnc_opt(K, n, cost): min total cost splitting a[0, n) into K groups; cost(j, i) = cost of group a[j, i). Needs monotone optimal split points.

```
ll best = dnc_opt(K, n, [&](int j, int i) { ll s = pre[i] - pre[j]; return s * s; });
```

Grid paths DP

Ctrl+Alt+T, Shift+Z cpgridpaths inside solve() O(nm) library/dp/grid_paths.cpp

needs n, m, grid. ways[r][c] = right/down paths from (0, 0) to (r, c) avoiding '#', mod MOD.

```
// answer: ways[n - 1][m - 1]; for a max-sum path use max(...) + value instead of the
    sum
```

Knapsack (0/1, unbounded, coin-change ways)

Ctrl+Alt+T, Shift+Q cpknap above solve() O(n W) library/dp/knapsack.cpp

knapsack_01(items, W) / knapsack_unbounded(items, W): max value, weight <= W; coin_ways(coins, X): combinations mod MOD.

```
vector<pair<int, ll>> items;  // {weight, value}
ll best = knapsack_01(items, W); ll ways = coin_ways(coins, X);
```

Knuth optimization (interval DP)

Ctrl+Alt+Y, Shift+Q cpknuth above solve() O(n^2) library/dp/knuth_optimization.cpp

knuth_opt(n, cost): dp[l][r] = min over l < m < r of dp[l][m] + dp[m][r] + cost(l, r), dp[i][i+1] = 0. Needs monotone opt.

```
ll best = knuth_opt(n, [&](int l, int r) { return pre[r] - pre[l]; });  // optimal
    merging of a[0, n)
```

LCS + edit distance

Ctrl+Alt+T, [cplcs above solve() O(nm) library/dp/lcs_edit.cpp

lcs(a, b) -> {length, one LCS}; edit_distance(a, b) = min insert / delete / replace operations.

```
auto [len, s] = lcs(a, b);
int d = edit_distance(a, b);
```

Li Chao tree (min of lines)

Ctrl+Alt+Y, Shift+L cplichao above solve() O(log range) per add / query library/dp/li_chao.cpp

LiChao lc(lo, hi); lc.add(k, m); lc.query(x) = min of k*x + m for integer x in [lo, hi). Nodes created on demand.

```
LiChao lc(-1e9, 1e9 + 1);
lc.add(k, m); ll best = lc.query(x);
```

LineContainer (convex hull trick, any order)

Ctrl+Alt+Y, L cplinecontainer above solve() O(log n) per add / query library/dp/line_container.cpp

LineContainer lc; lc.add(k, m); lc.query(x) = max of k*x + m. For a minimum add(-k, -m) and negate the query.

```
LineContainer lc;
lc.add(slope, intercept); ll best = lc.query(x);  // dp[i] = max_j (dp[j] + b[j] * a[i
    ]) in O(log n)
```

Longest increasing subsequence

Ctrl+Alt+T, Shift+L cplis above solve() O(n log n) library/dp/lis.cpp

vector<int> idx = lis(a): indices of one strictly increasing LIS, O(n log n). Length = idx.size().

```
vector<int> idx = lis(a);  // idx.size() = LIS length
```

Monotone convex hull trick (sorted slopes)

Ctrl+Alt+Y, Z cpmonocht above solve() O(1) amortized add, O(log n) query, O(1) amortized query_inc library/dp/monotone_cht.cpp

MonoCHT h; h.add(k, m) with non-increasing k; h.query(x) = min of k*x + m (binary search); h.query_inc(x) for non-decreasing x (pointer walk). For max: add(-k, -m), negate.

```
MonoCHT h; h.add(b[j], dp[j]);
dp[i] = h.query_inc(a[i]) + c[i];
```

Monotone minima (row minima when the argmin moves right)

Ctrl+Alt+U, M cpmonominima above solve() O((n + m) log n) evaluations of f library/dp/monotone_minima.cpp

monotone_minima(n, m, f): for each row i, the leftmost column j minimizing f(i, j), given that this argmin is non-decreasing in i (e.g. a Monge matrix).

```
vector<int> opt = monotone_minima(n, m, [&](int i, int j) { return cost(i, j); });
```

Range (interval) DP

Ctrl+Alt+T, - cprangedp inside solve() O(n^3) library/dp/range_dp.cpp

needs n, a. dp[l][r] over intervals by increasing length; example: merge adjacent piles, cost = merged sum.

```
// answer: dp[0][n - 1]; change the inner transition for your problem
```

Slope trick (make an array non-decreasing)

Ctrl+Alt+Y, Shift+S cpslope above solve() O(n log n) library/dp/slope_trick.cpp

make_non_decreasing(a): min total |a[i] - b[i]| over non-decreasing b. Max-heap of slope breakpoints. Strictly increasing: use a[i] - i.

```
ll cost = make_non_decreasing(a);
```

Subset convolution

Ctrl+Alt+U, Shift+S cpsubsetconv above solve() O(n^2 2^n) library/dp/subset_convolution.cpp

subset_convolution(a, b): c[S] = sum over T subset of S of a[T] * b[S \ T] (disjoint unions only), mod MOD. Sizes 2^n; n = 20 takes about 1 s and 176 MB.

```
vector<ll> c = subset_convolution(a, b);   // a.size() == b.size() == 1 << n
```

Subset sums with bitset

Ctrl+Alt+T, Shift+1 cpbitsetsum above solve() O(n MAXS / 64) library/dp/subset_sum_bitset.cpp

subset_sums<MAXS>(a): bitset with bit s set iff some subset of a sums to s (s < MAXS). 64x faster than a bool DP.

```
auto can = subset_sums<100001>(a);
if (can[target]) { ... }
```

Sum over subsets (SOS DP)

Ctrl+Alt+Y, U cpsos above solve() O(B 2^B) library/dp/sos_dp.cpp

sos_subsets(f, B): f[mask] becomes the sum of f[sub] over all submasks; sos_supersets: over supermasks. In place, size 2^B.

```
vector<ll> f(1 << B); /* f[mask] = count of numbers equal to mask */ sos_subsets(f, B);

// f[mask] now counts numbers x with (x & mask) == x
```

8.7 Math

Berlekamp-Massey (find a linear recurrence)

Ctrl+Alt+U, Shift+B cpbm above solve() O(n^2) library/math/berlekamp_massey.cpp

berlekamp_massey(s): shortest c with s[i] = c[0]*s[i-1] + c[1]*s[i-2] + ... mod MOD (prime). Feed 2L terms to find a recurrence of order L.

```
vector<ll> c = berlekamp_massey({1, 1, 2, 3, 5, 8, 13, 21});   // {1, 1}
ll x = linear_rec_nth(first_terms, c, n);   // with the Kitamasa template
```

Chinese remainder theorem (any moduli)

Ctrl+Alt+Y, 1 cpcrt above solve() O(log m) library/math/crt.cpp

crt(r1, m1, r2, m2) -> {r, lcm}: x = r (mod lcm) solves both congruences; {-1, -1} if none. Moduli need not be coprime.

```
auto [r, m] = crt(2, 3, 3, 5);  // x = 8 (mod 15)
// many congruences: fold left, feeding each result back in as (r1, m1); lcm must stay
     < ~4e18
```

Determinant mod p, matrix-tree theorem

Ctrl+Alt+U, Shift+D cpdet above solve() O(n^3) library/math/determinant.cpp

det_mod(A): determinant mod MOD (prime) by Gaussian elimination; count_spanning_trees(n, edges): Kirchhoff's matrix-tree theorem (multi-edges allowed). LGV lemma: det of path-count matrix.

```
ll d = det_mod(A);
ll t = count_spanning_trees(n, edges);   // edges: 0-indexed pairs
```

Discrete logarithm (baby-step giant-step)

Ctrl+Alt+U, L cpdlog above solve() O(sqrt(m)) time and memory library/math/discrete_log.cpp

discrete_log(a, b, m): smallest x >= 0 with a^x = b (mod m), or -1. Any m (not only primes), m <= 3e9.

```
ll x = discrete_log(2, 5, 13);   // 2^9 = 512 = 5 (mod 13) -> 9
```

Divisor sieve (count + sum of divisors)

Ctrl+Alt+T, Shift+2 cpdivsieve above solve() O(N log N) library/math/divisor_sieve.cpp

divisor_sieve(N) -> {cnt, sum}: number and sum of divisors of every x <= N by looping over multiples.

```
auto [cnt, sum] = divisor_sieve(1000000);
```

Du's sieve: prefix sums of phi and mu up to 1e10

Ctrl+Alt+U, D cpdusieve above solve() O(n^(2/3)) for the first query, memoised after library/math/du_sieve.cpp

DuSieve ds(n_max); ds.mu_sum(n) = mu(1)+...+mu(n) (exact), ds.phi_sum(n) = phi(1)+...+phi(n) mod MOD.

```
DuSieve ds(10'000'000'000LL);
ll M = ds.mu_sum(n), Phi = ds.phi_sum(n);
```

Euler's totient

Ctrl+Alt+Y, Shift+E cpphi above solve() O(sqrt n) single, O(N log log N) sieve library/math/totient.cpp

phi(n) by trial division; phi_sieve(N) = phi of every x <= N. phi(n) counts 1 <= k <= n with gcd(k, n) = 1.

```
ll p = phi(n);
auto ph = phi_sieve(1000000);  // a^phi(m) = 1 (mod m) when gcd(a, m) = 1
```

Extended GCD + inverse for any modulus

Ctrl+Alt+T, X cpextgcd above solve() O(log min(a, b)) library/math/ext_gcd.cpp

g = ext_gcd(a, b, x, y) with a*x + b*y = g; inv_mod(a, m) for gcd(a, m) = 1, else -1.

```
ll x, y, g = ext_gcd(a, b, x, y);  // a*x + b*y = g
inv_mod(a, m);  // -1 if gcd(a, m) != 1
```

Factorize + divisors (trial division)

Ctrl+Alt+T, Shift+P cpfactor above solve() O(sqrt x) library/math/factorize.cpp

factorize(x) -> {prime, exponent}, divisors(x) sorted. O(sqrt x), fine up to ~1e12.

```
for (auto [p, e] : factorize(x)) { ... }
vector<ll> d = divisors(x);
```

FFT (exact integer convolution)

Ctrl+Alt+Y, F cpfft above solve() O((n + m) log(n + m)) library/math/fft.cpp

multiply(a, b): convolution c[k] = sum a[i] b[k - i] with complex FFT; exact while results stay below ~1e15.

```
vector<ll> c = multiply(a, b);  // polynomial product, counting pairs with a given sum,
    ...
```

Floor-division blocks

Ctrl+Alt+Y, Shift+F cpfloorblocks above solve() O(sqrt n) library/math/floor_blocks.cpp

for_each_floor_block(n, f): f(l, r, q) for every maximal [l, r] where n / i == q; only O(sqrt n) blocks.

```
ll s = 0; for_each_floor_block(n, [&](ll l, ll r, ll q) { s += q * (r - l + 1); });  //
    sum of n / i
// sum of divisors of 1..n: q * (l + r) * (r - l + 1) / 2 per block
```

Grundy numbers (Sprague-Grundy)

Ctrl+Alt+Y, G cpgrundy above solve() O(N * moves) library/math/grundy.cpp

grundy_numbers(N, moves): g[s] = mex of g over moves(s) (moves must go to smaller states). XOR of games != 0 -> first player wins.

```
auto g = grundy_numbers(N, [&](int s) { vector<int> to; for (int k : {1, 3, 4}) if (s
    >= k) to.push_back(s - k); return to; });
int x = 0; for (int pile : piles) x ^= g[pile]; bool first_wins = x != 0;
```

Inclusion-exclusion (divisible by any)

Ctrl+Alt+Y, 3 cpincexc above solve() O(2^k k) library/math/inclusion_exclusion.cpp

count_divisible_by_any(N, ds): how many x in [1, N] are divisible by at least one of ds (k <= ~20). Odd subsets add, even subtract.

```
ll c = count_divisible_by_any(N, {2, 3, 5});
// coprime to m: N - count_divisible_by_any(N, distinct primes of m)
```

Lagrange interpolation

Ctrl+Alt+U, I cplagrange above solve() O(k) consecutive points (plus O(log MOD)), O(k^2) general points library/math/lagrange.cpp

lagrange_consecutive(y, x): the degree < k polynomial with P(i) = y[i] for i = 0..k-1, evaluated at any x; lagrange_general(xs, ys, x) for arbitrary distinct points. Mod MOD (prime).

```
ll s = lagrange_consecutive({0, 1, 5, 14}, 1'000'000'000);   // 0^2 + 1^2 + ... +
    10^9^2 mod MOD
ll v = lagrange_general(xs, ys, x);
```

Matrix exponentiation

Ctrl+Alt+T, Shift+X cpmatrix above solve() O(k^3 log e) library/math/matrix.cpp

Mat R = mat_pow(A, e), mat_mul(A, B), entries mod MOD. Linear recurrences in O(k^3 log e).

```
Mat M = {{1, 1}, {1, 0}};
ll fib_n = mat_pow(M, n)[0][1];
```

Miller-Rabin + Pollard rho (numbers up to 1e18)

Ctrl+Alt+Y, X cppollard above solve() ~O(n^(1/4) log n) per factorization library/math/pollard_rho.cpp

BigFactor::is_prime(n) (deterministic below 2^64), BigFactor::factor(n) -> prime factors with repeats, unsorted.

```
bool p = BigFactor::is_prime(n);
auto f = BigFactor::factor(n); sort(f.begin(), f.end());
```

Mint (modular integer)

Ctrl+Alt+T, Shift+M cpmint above solve() O(1) per op, O(log MOD) for / and inv library/math/mint.cpp

mint x = 5; x += 3; x *= x.inv(); x.pow(e); cout << x. Uses the template's MOD (must be prime for /).

```
mint x = 5;
x = x * 3 - 1; x /= 2; x.pow(10); cout << x;
```

Mobius function sieve

Ctrl+Alt+Y, Shift+M cpmobius above solve() O(N) library/math/mobius.cpp

mobius_sieve(N) = mu[0..N] by a linear sieve. Key identity: sum over d | g of mu(d) = [g == 1] (count coprime pairs, etc.).

```
auto mu = mobius_sieve(N);
// pairs (a, b) <= N with gcd 1: sum over d of mu[d] * (N / d) * (N / d)
```

Modular power + inverse

Ctrl+Alt+T, M cpmodpow above solve() O(log e) library/math/modpow.cpp

modpow(a, e, m = MOD), modinv(a, m = MOD) for prime m. Uses the template's MOD.

```
modpow(2, n);  // 2^n mod MOD
modinv(x);    // x^-1 mod MOD
```

N-th term of a linear recurrence (Kitamasa)

Ctrl+Alt+U, K cpkitamasa above solve() O(L^2 log n) library/math/linear_recurrence.cpp

linear_rec_nth(a, c, n): a[n] for a[i] = c[0]*a[i-1] + ... + c[L-1]*a[i-L], given a[0..L-1]; mod MOD. Pairs with Berlekamp-Massey.

```
ll f = linear_rec_nth({0, 1}, {1, 1}, 1'000'000'000'000'000'000LL);   // Fibonacci
```

nCr / factorials mod p

Ctrl+Alt+T, N cpncr above solve() O(N) init, O(1) per query library/math/ncr.cpp

init_comb(N) once, then nCr, nPr, catalan, stars_and_bars, fact[i], inv_fact[i]. Uses the template's MOD.

```
init_comb(200000);
nCr(n, r); nPr(n, r); catalan(n); fact[n];
```

NTT (convolution mod 998244353)

Ctrl+Alt+Y, Shift+T cpntt above solve() O((n + m) log(n + m)) library/math/ntt.cpp

ntt_multiply(a, b): convolution with every coefficient mod 998244353, exact for any size up to 2^23.

```
vector<ll> c = ntt_multiply(a, b);
```

Pascal triangle (nCr for any modulus)

Ctrl+Alt+Y, Shift+N cppascal above solve() O(N^2) library/math/pascal.cpp

pascal(N, m)[n][r] = C(n, r) mod m for n <= N; works when m is not prime or is 1e18-ish (then keep N small).

```
auto C = pascal(60, 1e18);  // exact binomials up to C(60, 30) ~ 1.2e17
```

Polynomial toolkit mod 998244353 (inverse, divmod, log, exp, sqrt, pow, multipoint)

Ctrl+Alt+U, Shift+P cppoly above solve() O(n log n) for mul / inv / divmod / log / exp / sqrt / pow; O(n log^2 n) for multipoint and interpolation library/math/poly.cpp

Poly = vector<ll>. poly_mul, poly_inv(a, n), poly_divmod(a, b), poly_log / poly_exp / poly_sqrt (a, n), poly_pow(a, k, n), poly_eval_many(a, xs), poly_interpolate(xs, ys). Self-contained NTT.

```
Poly b = poly_inv(a, n);   // a * b = 1 mod x^n, needs a[0] != 0
auto [q, r] = poly_divmod(a, b);  Poly e = poly_exp(a, n);  vector<ll> v =
    poly_eval_many(a, xs);
```

Prime counting and prime sums up to 1e11 (Lucy)

Ctrl+Alt+U, P cpprimecount above solve() O(n^(3/4)) time, O(sqrt n) memory library/math/prime_count.cpp

prime_count(n) = number of primes <= n (exact); prime_sum(n) = sum of primes <= n mod MOD.

```
ll pi = prime_count(100'000'000'000LL);   // 4118054813
ll s = prime_sum(n);
```

Primitive root

Ctrl+Alt+U, G cpprimroot above solve() O(sqrt(m) + answer * (number of prime factors of phi) * log m) library/math/primitive_root.cpp

primitive_root(m): smallest generator g of (Z/mZ)* (exists for m = 1, 2, 4, p^k, 2p^k), else -1. m <= 3e9.

```
ll g = primitive_root(998244353);   // 3
```

Sieve (primes + smallest prime factor)

Ctrl+Alt+T, P cpsieve above solve() O(N) init, O(log x) per factorization library/math/sieve.cpp

init_sieve(N); primes, spf[x], is_prime(x), factor_small(x) -> {p, e} in O(log x) for x <= N.

```
init_sieve(10000000);
is_prime(x); primes; factor_small(x);  // x <= N
```

Square root modulo a prime (Tonelli-Shanks)

Ctrl+Alt+U, Q cpsqrtmod above solve() O(log^2 p) library/math/sqrt_mod.cpp

sqrt_mod(a, p): some x with x*x = a (mod p) for prime p (the other root is p - x), or -1 if a is not a square.

```
ll x = sqrt_mod(10, 13);   // 6 or 7: 6*6 = 36 = 10 (mod 13)
```

Stern-Brocot: simplest fraction in an interval, best approximation

Ctrl+Alt+U, B cpsternbrocot above solve() O(log max) (a Euclid-like walk down the Stern-Brocot tree) library/math/stern_brocot.cpp

simplest_between(a, b, c, d): p/q with the smallest q (then p) and a/b < p/q < c/d (0 <= a/b < c/d); best_approx(x, y, N): p/q closest to x/y with q <= N.

```
auto [p, q] = simplest_between(1, 3, 1, 2);   // 2/5
auto [p2, q2] = best_approx(314159, 100000, 100);   // 311/99
```

Stirling numbers (both kinds) and Bell numbers

Ctrl+Alt+U, S cpstirling above solve() O(N^2) time and memory library/math/stirling.cpp

init_stirling(N): S2[n][k] (partitions of n items into k nonempty sets), S1[n][k] (permutations of n with k cycles), bell[n]; all mod MOD.

```
init_stirling(1000);
S2[n][k]; S1[n][k]; bell[n];
```

Walsh-Hadamard transform: XOR / AND / OR convolution

Ctrl+Alt+U, W cpfwht above solve() O(n log n) for n = padded size library/math/fwht.cpp

xor_convolution(a, b), and_convolution(a, b), or_convolution(a, b): c[k] = sum of a[i]*b[j] over i^j = k (i&j, i|j) mod MOD. Sizes padded to a power of two.

```
vector<ll> c = xor_convolution(a, b);
```

XOR basis (linear basis over GF(2))

Ctrl+Alt+Y, Shift+X cpxorbasis above solve() O(B) per operation, B = 62 bits library/math/xor_basis.cpp

XorBasis xb; xb.insert(x) (false if x was already representable); xb.can_make(x); xb.max_xor(); 2^xb.dim distinct subset XORs.

```
XorBasis xb; for (auto x : a) xb.insert(x);
ll best = xb.max_xor(); bool ok = xb.can_make(target);
```

8.8 Strings

Aho-Corasick (many patterns at once)

Ctrl+Alt+Y, Shift+A cpaho above solve() O(total pattern length * 26 + |text|) library/string/aho_corasick.cpp

AhoCorasick ac; id = ac.add(p) per pattern; ac.build(); ac.occurrences(text)[id] = matches of that pattern. a-z only.

```
AhoCorasick ac; vector<int> id; for (auto& p : pats) id.push_back(ac.add(p)); ac.build
    ();
auto occ = ac.occurrences(text); ll hits = occ[id[i]];  // or walk: s = ac.go(s, ch),
    ac.out[s] = matches ending here
```

Binary trie (maximum XOR)

Ctrl+Alt+T, ` cpxortrie above solve() O(B) per operation, B = 30 (set B = 61 for 64-bit values) library/string/xor_trie.cpp

XorTrie t; t.insert(x), t.erase(x) (x must be present), t.max_xor(x) = max over stored y of x ^ y (non-empty). 30 bits.

```
XorTrie t; t.insert(0);  // prefix XORs: best subarray XOR = max over i of t.max_xor(
    pref[i])
ll best = t.max_xor(x);
```

KMP (prefix function + pattern search)

Ctrl+Alt+T, Shift+K cpkmp above solve() O(n + m) library/string/kmp.cpp

prefix_function(s) -> pi; find_occurrences(text, pat) -> start index of every match.

```
vector<int> starts = find_occurrences(text, pat);
vector<int> pi = prefix_function(s);
```

Lyndon factorization (Duval) and minimal rotation

Ctrl+Alt+U, Y cplyndon above solve() O(n) library/string/lyndon.cpp

duval(s): start indices of the Lyndon factorization s = w1 w2 ... wk with w1 >= w2 >= ... (each wi strictly smaller than all its rotations); min_rotation(s): start index of the lexicographically smallest rotation.

```
vector<int> starts = duval(s);
int k = min_rotation(s); string canon = s.substr(k) + s.substr(0, k);
```

Manacher (all palindromes)

Ctrl+Alt+Y, 6 cpmanacher above solve() O(n) library/string/manacher.cpp

manacher(s) -> {d1, d2}: d1[i] = odd palindromes centred at i, d2[i] = even ones centred before i; longest_palindrome(s) -> {start, len}.

```
auto [start, len] = longest_palindrome(s);
// number of palindromic substrings = sum(d1) + sum(d2)
```

Palindromic tree (eertree)

Ctrl+Alt+U, Shift+E cpeertree above solve() O(n * alphabet) memory, O(n) amortized time library/string/eertree.cpp

Eertree t; t.add(c) for each char (lowercase). Nodes 2.. are the distinct palindromic substrings (len, link = longest proper palindromic suffix); t.last = longest palindromic suffix of the current prefix. occurrences() counts each palindrome's occurrences.

```
Eertree t; for (char c : s) t.add(c);
int distinct = t.t.size() - 2; vector<ll> occ = t.occurrences();
```

String hashing (mod 2^61-1)

Ctrl+Alt+T, H cpstrhash above solve() O(n) build, O(1) per substring library/string/string_hash.cpp

StrHash h(s); h.get(l, r) = hash of s[l, r). Random base, hack-safe; same base for every string.

```
StrHash h(s);
if (h.get(l1, r1) == h.get(l2, r2)) { /* s[l1, r1) == s[l2, r2) */ }
```

Suffix array + LCP

Ctrl+Alt+Y, 4 cpsa above solve() O(n log n) library/string/suffix_array.cpp

suffix_array(s): sa[i] = start of the i-th smallest suffix; lcp_array(s, sa)[i] = LCP of suffixes sa[i] and sa[i + 1].

```
auto sa = suffix_array(s); auto lcp = lcp_array(s, sa);
// distinct substrings = n(n+1)/2 - sum(lcp)
```

Suffix automaton

Ctrl+Alt+Y, 5 cpsam above solve() O(n * 26) library/string/suffix_automaton.cpp

SuffixAutomaton sam(s): smallest automaton of all substrings. sam.count_distinct(), sam.contains(t); states hold len / link / next.

```
SuffixAutomaton sam(s);
ll d = sam.count_distinct(); bool in = sam.contains(t);
```

Trie

Ctrl+Alt+T, Shift+T cptrie above solve() O(|s|) per operation library/string/trie.cpp

Trie t; t.insert(s); t.count(s) = copies of s; t.prefix_count(p) = words starting with p. a-z only.

```
Trie t;
t.insert(s); t.count(s); t.prefix_count(p);
```

Z-function

Ctrl+Alt+T, Z cpz above solve() O(n) library/string/z_function.cpp

z_function(s): z[i] = longest common prefix of s and s[i..]; z[0] = 0.

```
vector<int> z = z_function(pat + '#' + text);  // z[i] == |pat| -> match
```

8.9 Geometry

Closest pair of points

Ctrl+Alt+Y, 9 cpclosest above solve() O(n log n) library/geometry/closest_pair.cpp

closest_pair(pts) = smallest squared distance between two of the points (n >= 2), sweep by x with a set by y. Needs cppoint.

```
ll d2 = closest_pair(pts);
```

Convex hull (monotone chain)

Ctrl+Alt+T, Shift+V cphull above solve() O(n log n) library/geometry/convex_hull.cpp

convex_hull(pts) -> counter-clockwise hull without collinear points. Pt = pair<ll, ll>.

```
vector<Pt> hull = convex_hull(pts);
```

Geometry primitives (integer points)

Ctrl+Alt+T, Shift+3 cppoint above solve() O(1) library/geometry/point.cpp

Point {x, y} with + - * dot cross norm2; orient(a, b, c) = +1 left turn, -1 right, 0 collinear; on_segment(p, a, b). Exact.

```
Point a{1, 2}, b, c; cin >> b >> c;
int o = orient(a, b, c); bool on = on_segment(p, a, b); ll d2 = (a - b).norm2();
```

Half-plane intersection

Ctrl+Alt+U, 4 cphalfplane above solve() O(n log n) library/geometry/half_plane.cpp

half_plane_intersection(hs, B): hs[i] = {ax, ay, bx, by} keeps the points LEFT of the directed line a->b. Returns the intersection polygon (CCW, long double) clipped to the box [-B, B]^2, or empty if it is empty.

```
vector<array<long double, 4>> hs = {{0, 0, 1, 0}, {1, 0, 0, 1}, {0, 1, 0, 0}};   // a
    triangle
auto poly = half_plane_intersection(hs, 1e9);
```

Minimum enclosing circle (Welzl, randomized)

Ctrl+Alt+U, Shift+M cpmincircle above solve() O(n) expected library/geometry/min_enclosing_circle.cpp

min_enclosing_circle(pts): the smallest circle {cx, cy, r} containing all points (long double). Random shuffle + incremental construction.

```
auto [cx, cy, r] = min_enclosing_circle(pts);   // pts: vector<pair<long double, long
    double>>
```

Polygon: area, point inside, lattice points

Ctrl+Alt+Y, 8 cppolygon above solve() O(n) library/geometry/polygon.cpp

area2(poly) = 2 * signed area (> 0 counter-clockwise); in_polygon(poly, p): 1 inside, 0 boundary, -1 outside; boundary_points. Needs cppoint.

```
ll twice = llabs(area2(poly)); int where = in_polygon(poly, p);
ll interior = (twice - boundary_points(poly) + 2) / 2;  // Pick's theorem
```

Rectangles (intersection, union of two)

Ctrl+Alt+T, Shift+4 cprect above solve() O(1) library/geometry/rectangles.cpp

Rect {x1, y1, x2, y2} (x1 < x2, y1 < y2): area(), intersect_area(a, b), union_area(a, b). USACO bronze staples.

```
Rect a{0, 0, 4, 3}, b{2, 1, 6, 5};
ll both = intersect_area(a, b), either = union_area(a, b);
```

Rotating calipers (diameter and width of a convex polygon)

Ctrl+Alt+U, 5 cpcalipers above solve() O(n) library/geometry/rotating_calipers.cpp

For a convex polygon h (CCW, no three collinear, e.g. from the convex hull template): hull_diameter2(h) = max squared distance between two vertices (exact); hull_width(h) = min distance between two parallel lines enclosing it.

```
ll d2 = hull_diameter2(hull);   // hull: vector<pair<ll, ll>>
long double w = hull_width(hull);
```

Segment intersection

Ctrl+Alt+Y, 7 cpseginter above solve() O(1) library/geometry/segment_intersection.cpp

segments_intersect(a, b, c, d): closed segments ab and cd share a point (touching and overlapping count). Needs cppoint.

```
bool hit = segments_intersect(a, b, c, d);
```

8.10 Flows & matching

Bipartite matching (Hopcroft-Karp)

Ctrl+Alt+Y, Shift+H cphopcroft above solve() O(E sqrt V) library/flow/hopcroft_karp.cpp

HopcroftKarp hk(nl, nr); hk.add_edge(u, v) with u on the left, v on the right; hk.max_matching(); hk.match_l[u] = partner or -1.

```
HopcroftKarp hk(nl, nr); hk.add_edge(u, v);
int m = hk.max_matching();  // Konig: min vertex cover = m; max independent set = nl +
    nr - m
```

Bipartite minimum vertex cover / maximum independent set (Konig)

Ctrl+Alt+U, V cpkonig above solve() O(V E) (augmenting paths) library/flow/min_vertex_cover.cpp

konig(nl, nr, adj): adj[left] = right neighbours. Returns {cover_left, cover_right}: a minimum vertex cover, whose size equals the maximum matching; the complement is a maximum independent set.

```
auto [cl, cr] = konig(nl, nr, adj);   // vertices in the cover (bool per side)
```

Global minimum cut (Stoer-Wagner)

Ctrl+Alt+U, Shift+A cpstoerwagner above solve() O(n^3) library/flow/stoer_wagner.cpp

stoer_wagner(w): the minimum total weight of edges whose removal disconnects the graph, w = symmetric n x n weight matrix (n >= 2). No source or sink needed.

```
ll cut = stoer_wagner(w);
```

Gomory-Hu tree (all-pairs minimum cut)

Ctrl+Alt+U, A cpgomoryhu above solve() n - 1 max flows (Dinic) library/flow/gomory_hu.cpp

gomory_hu(n, edges): a tree par[v], w[v] (v >= 1) on 0..n-1 for an undirected capacitated graph; the min cut between any u and v equals the smallest w on the tree path (Gusfield's algorithm, n - 1 max flows).

```
auto [par, w] = gomory_hu(n, edges);   // edges: {u, v, capacity}
```

Hungarian algorithm (min-cost assignment)

Ctrl+Alt+U, Shift+H cphungarian above solve() O(n^2 m) library/flow/hungarian.cpp

hungarian(a): a is n x m with n <= m; assigns each row a distinct column minimizing the total cost. Returns {cost, col_of_row}. For maximum weight, negate the costs.

```
auto [cost, col] = hungarian(a);   // row i -> column col[i]
```

Max flow (Dinic) + min cut

Ctrl+Alt+Y, Y cpdinic above solve() O(V^2 E) worst case, O(E sqrt V) on unit-capacity graphs library/flow/dinic.cpp

Dinic g(n); g.add_edge(u, v, cap); ll f = g.max_flow(s, t); after it g.min_cut_side(v) = v on the source side of a min cut.

```
Dinic g(n); int e = g.add_edge(u, v, cap);
ll f = g.max_flow(s, t); ll flow_on_e = g.flow_on(e);
```

Max flow with lower bounds

Ctrl+Alt+Y, Shift+G cpflowlb above solve() one or two Dinic runs library/flow/lower_bounds.cpp

LowerBoundFlow lb(n); lb.add_edge(u, v, low, high); lb.max_flow(s, t) = max s-t flow meeting every bound, -1 if none. Needs cpdinic.

```
LowerBoundFlow lb(n); lb.add_edge(u, v, 1, 3);
ll f = lb.max_flow(s, t);  // -1: the bounds cannot be met
```

Maximum matching in a general graph (Edmonds' blossom)

Ctrl+Alt+U, Shift+G cpblossom above solve() O(n^3) library/flow/blossom.cpp

blossom_matching(adj): match[v] = partner or -1, for any undirected graph (odd cycles allowed). Matching size = number of v with match[v] != -1, divided by 2.

```
vector<int> mate = blossom_matching(adj);
```

Min-cost max-flow

Ctrl+Alt+Y, Shift+Y cpmcmf above solve() O(flow * V * E) worst case, far less in practice library/flow/mcmf.cpp

MCMF g(n); g.add_edge(u, v, cap, cost); auto [flow, cost] = g.run(s, t, limit). Shortest paths by SPFA, so negative costs are fine.

```
MCMF g(n); g.add_edge(u, v, 1, w);
auto [flow, cost] = g.run(s, t);  // cheapest among maximum flows (or among flows of
    size limit)
```

8.11 Misc

De Bruijn sequence

Ctrl+Alt+U, 9 cpdebruijn above solve() O(k^n) library/misc/de_bruijn.cpp

de_bruijn(k, n): a cyclic sequence of length k^n over {0..k-1} in which every length-n word appears exactly once (read cyclically). Built from Lyndon words (FKM algorithm). Shortest string containing all words: the sequence + its first n-1 symbols.

```
vector<int> seq = de_bruijn(2, 3);   // 0 0 0 1 0 1 1 1
```

Interactive problem helpers

Ctrl+Alt+T, Shift+I cpinteractive inside solve() – library/misc/interactive.cpp

ask(i, j) prints "? i j", flushes, reads the reply (exits on -1); answer(x) prints "! x".

```
int r = ask(i, j);
answer(x);
```

Random generator

Ctrl+Alt+T, Shift+R cprng above solve() O(1) library/misc/random.cpp

rnd(l, r) uniform in [l, r]; shuffle(v.begin(), v.end(), rng). Seeded by time.

```
ll x = rnd(1, 100);
shuffle(v.begin(), v.end(), rng);
```

Random hashing of sets and multisets (XOR / sum hashing)

Ctrl+Alt+U, X cpxorhash above solve() O(1) per label library/misc/xor_hash.cpp

RandHash h; h(x) = a random 64-bit label for value x. Multisets are equal (whp) iff the sums of labels match; "every value appears an even number of times" iff the XOR of labels is 0. With prefix sums / XORs this answers range questions in O(1).

```
RandHash h; vector<uint64_t> px(n + 1); for (int i = 0; i < n; i++) px[i + 1] = px[i]
    ^ h(a[i]);
bool all_even = (px[r] ^ px[l]) == 0;   // range [l, r)
```

Simulated annealing skeleton

Ctrl+Alt+U, Z cpanneal above solve() (seconds) x (cost of energy + mutate) library/misc/annealing.cpp

anneal(s, energy, mutate, seconds, t0, t1): minimize energy(state) by random local moves; worse moves are accepted with probability exp(-delta / T), T cooling from t0 to t1 geometrically. Returns {best state, best energy}. For heuristic / output-only problems.

```
auto [best, e] = anneal(start, [&](const State& s) { return cost(s); }, [&](State s,
    mt19937_64& rng) { /* small change */ return s; }, 1.8, 10.0, 0.01);
```